Bachelor's Degree Program in Software Engineering



Prof. Dr. Paulo Ditarso Maciel Jr. (paulo.maciel@ifpb.edu.br)

October 19, 2026

Operating Systems

Based on material from the book *Operating Systems: Three Easy Pieces*.

```
$ git clone https://github.com/pdmjr/ostep_slides.git
```

Disclaimer[1]

- This presentation is not intended to dive deeply into any topic.
- It is essential to read the books and materials provided!
- What is the purpose of this presentation?



[1] All figures used in this presentation are publicly available on websites, scientific papers or are licensed under the *Creative Commons*.

INSTITUTO FEDERAL
Paraíba

Agenda

1. A Dialogue on the Book
2. Introduction
3. A Dialogue on Virtualization
4. Processes
5. Process API
6. Direct Execution
7. CPU Scheduling
8. Multi-Level Feedback Queue
9. Lottery Scheduling
10. Multiprocessor Scheduling
11. Summary Dialogue on CPU Virtualization
12. A Dialogue on Memory Virtualization
13. The Abstraction: Address Spaces
14. Interlude: Memory API
15. Mechanism: Address Translation
16. Segmentation
17. Free-Space Management

A Dialogue on the Book

A Dialogue on the Book

The dialogue introduces the book and explains the origin of its title, inspired by Richard Feynman's *Six Easy Pieces*.

The book is organized around three fundamental ideas: **virtualization**, **concurrency**, and **persistence**. Through these topics, it explores how operating systems manage processors, memory, storage, virtual machines, and even some aspects of distributed systems.

The dialogue also emphasizes an effective approach to learning operating systems: attending classes, reviewing the material regularly, completing homework, and especially developing projects that involve writing real code to solve real problems.

Finally, the dialogue explains that these conversations are intended to encourage reflection on complex concepts and help the reader reason about the ideas presented throughout the book.

Introduction

What Happens When a Program Runs?

- A running program executes instructions.
- The processor repeatedly **fetches**, **decodes**, and **executes** instructions.
- Programs assume an orderly, sequential execution model.
- The operating system works underneath this simple model to make the machine easier to use.

Key idea

The OS hides much of the complexity involved in sharing and managing hardware resources.

What Does an Operating System Do?

- Makes it easier to run programs and use hardware resources.
- **Virtualizes** physical resources such as the CPU, memory, and storage.
- Provides interfaces through **system calls**.
- Acts as a **resource manager** for shared system resources.

The Crux of the Problem

How does the operating system virtualize resources efficiently, and what hardware support is needed?

Virtualizing the CPU

- The example program repeatedly waits and prints a user-provided string.
- A single program running alone is straightforward.
- The interesting behavior appears when several copies execute at once.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <sys/time.h>
4  #include <assert.h>
5  #include "common.h"
6
7  int
8  main(int argc, char *argv[])
9  {
10     if (argc != 2) {
11         fprintf(stderr, "usage: cpu <string>\n");
12         exit(1);
13     }
14     char *str = argv[1];
15     while (1) {
16         Spin(1);
17         printf("%s\n", str);
18     }
19     return 0;
20 }
```

Figure 2.1: **Simple Example: Code That Loops And Prints (cpu.c)**

Source: chapter page 3.

The Illusion of Many CPUs

```
prompt> ./cpu A & ./cpu B & ./cpu C & ./cpu D &
[1] 7353
[2] 7354
[3] 7355
[4] 7356
A
B
D
C
A
B
D
C
A
...
```

Figure 2.2: **Running Many Programs At Once**

Source: chapter page 4.

- Multiple programs appear to run simultaneously even on one processor.
- The OS, with hardware support, creates the illusion of many **virtual CPUs**.
- The OS also needs policies to decide which program should run at a given time.

Virtualizing Memory

- Memory can be viewed as an array of bytes addressed by the processor.
- Programs continuously access memory for instructions and data.
- The example allocates memory with *malloc()*, initializes it, and repeatedly updates it.

```
1  #include <unistd.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include "common.h"
5
6  int
7  main(int argc, char *argv[])
8  {
9      int *p = malloc(sizeof(int));                    // a1
10     assert(p != NULL);
11     printf("(%d) address pointed to by p: %p\n",
12            getpid(), p);                             // a2
13     *p = 0;                                          // a3
14     while (1) {
15         Spin(1);
16         *p = *p + 1;
17         printf("(%d) p: %d\n", getpid(), *p);        // a4
18     }
19     return 0;
20 }
```

Figure 2.3: **A Program That Accesses Memory (`mem.c`)**

Source: chapter page 5.

Private Virtual Address Spaces

```
prompt> ./mem & ./mem &
[1] 24113
[2] 24114
(24113) address pointed to by p: 0x200000
(24114) address pointed to by p: 0x200000
(24113) p: 1
(24114) p: 1
(24114) p: 2
(24113) p: 2
(24113) p: 3
(24114) p: 3
(24113) p: 4
(24114) p: 4
...
```

Figure 2.4: **Running The Memory Program Multiple Times**

Source: chapter page 6.

- Two processes can appear to use the same address independently.
- Each process sees its own private **virtual address space**.
- The OS maps each virtual address space onto shared physical memory.
- One process normally does not affect another process's address space.

Concurrency

- Concurrency refers to problems that arise when multiple activities proceed at the same time.
- Modern programs may contain multiple threads sharing the same memory space.
- The example creates two worker threads that increment a shared counter.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "common.h"
4  #include "common_threads.h"
5
6  volatile int counter = 0;
7  int loops;
8
9  void *worker(void *arg) {
10     int i;
11     for (i = 0; i < loops; i++) {
12         counter++;
13     }
14     return NULL;
15 }
16
17 int main(int argc, char *argv[]) {
18     if (argc != 2) {
19         fprintf(stderr, "usage: threads <value>\n");
20         exit(1);
21     }
22     loops = atoi(argv[1]);
23     pthread_t p1, p2;
24     printf("Initial value : %d\n", counter);
25
26     Pthread_create(&p1, NULL, worker, NULL);
27     Pthread_create(&p2, NULL, worker, NULL);
28     Pthread_join(p1, NULL);
29     Pthread_join(p2, NULL);
30     printf("Final value   : %d\n", counter);
31     return 0;
32 }
```

Figure 2.5: **A Multi-threaded Program (`threads.c`)**

Source: chapter page 7.

Why Concurrency Is Difficult

- With 1,000 increments per thread, the expected final value can be observed.
- With larger loop counts, the program may produce incorrect and different results.
- Incrementing a shared counter requires multiple machine instructions.
- Because the operation is not atomic, thread interleavings can produce unexpected outcomes.

The Crux of the Problem

How can we build correct programs when many threads execute concurrently in the same memory space?

Persistence

- Main memory is volatile: data can disappear after power loss or a crash.
- Persistent storage keeps data for the long term.
- The file system manages persistent data on storage devices.
- Applications use system calls such as *`open()`*, *`write()`*, and *`close()`*.

```
1  #include <stdio.h>
2  #include <unistd.h>
3  #include <assert.h>
4  #include <fcntl.h>
5  #include <sys/types.h>
6
7  int main(int argc, char *argv[]) {
8      int fd = open("/tmp/file",
9                    O_WRONLY|O_CREAT|O_TRUNC,
10                   S_IRWXU);
11     assert(fd > -1);
12     int rc = write(fd, "hello world\n", 12);
13     assert(rc == 12);
14     close(fd);
15     return 0;
16 }
```

Figure 2.6: **A Program That Does I/O (`io.c`)**

Source: chapter page 10.

Reliable Persistent Storage

- File systems must determine where data is stored and maintain metadata about it.
- They issue I/O requests to underlying storage devices.
- For performance, writes may be delayed and grouped.
- Techniques such as **journaling** and **copy-on-write** help systems recover from failures.

The Crux of the Problem

How can the OS store data persistently with correctness, performance, and reliability?

Operating System Design Goals

- **Abstraction:** make the system convenient and manageable.
- **Performance:** minimize time and space overheads.
- **Protection and isolation:** prevent applications from harming one another or the OS.
- **Reliability:** keep the operating system running correctly.
- Additional goals may include energy efficiency, security, and mobility.

A Brief History of Operating Systems

1. **Early systems:** OS as libraries of common functions; batch processing.
2. **Protection:** system calls, user mode, kernel mode, and controlled privilege transitions.
3. **Multiprogramming:** several jobs in memory, rapid switching, better CPU utilization.
4. **UNIX:** simple, powerful abstractions and tools; strong influence on later systems.
5. **Modern era:** mature OS ideas returned to personal computers and later to mobile systems.

Chapter Summary

Three Major Themes

1. **Virtualization** of the CPU and memory.
2. **Concurrency** and correct coordination among simultaneous activities.
3. **Persistence** through devices and file systems.

- The OS provides abstractions and interfaces that make hardware easier to use.
- It manages resources while pursuing performance, protection, and reliability.

Virtualization

A Dialogue on Virtualization

A Dialogue on Virtualization

The dialogue introduces **virtualization** through the metaphor of a single physical peach being shared among several people, while each person has the illusion of having their own peach.

The same idea is applied to computing: a physical resource, such as a CPU, can be shared among multiple applications while each application behaves as if it had its own dedicated CPU.

The key idea is that the operating system creates this illusion by managing and sharing physical resources efficiently. In the case of the CPU, virtualization makes one physical processor appear as multiple virtual CPUs to running applications.

Thus, virtualization allows several programs to use the same hardware while giving each one the impression of exclusive access to the resource.

Processes

The Process Abstraction

- A **process** is a running program.
- A program stored on disk is passive:
  - instructions;
  - static data.
- The operating system transforms a program into an active process.
- Modern systems execute many processes seemingly at the same time.

Crux of the Problem

How can the OS provide the illusion of many CPUs when only a few physical CPUs are available?

Virtualizing the CPU

- The OS creates the illusion of many virtual CPUs.
- It runs one process, stops it, runs another, and so forth.
- This technique is called **time sharing**.
- CPU time is shared among concurrent processes.
- The main cost of sharing is reduced performance for each process.

Time Sharing

A resource is used by one entity for a period of time and then assigned to another.

Mechanisms and Policies

- CPU virtualization requires two important OS concepts:

Mechanisms

Low-level methods that implement functionality.

Example: stopping one process and starting another through a **context switch**.

Policies

Algorithms that determine which decision the OS should make.

Example: deciding which process should run next.

Mechanism = How? **Policy = Which?**

What Defines a Process?

A process can be described through its **machine state**.

- **Address space**
  - program instructions;
  - program data.
- **CPU registers**
  - general-purpose registers;
  - program counter (PC);
  - stack pointer;
  - frame pointer.
- **I/O information**
  - open files and other resources.

Process API

A modern operating system provides interfaces to manage processes.

- **Create**: create a new process.
- **Destroy**: terminate a process.
- **Wait**: wait for a process to finish.
- **Miscellaneous Control**: suspend and resume processes.
- **Status**: obtain information about a process.

From Program to Process

- Programs initially reside on persistent storage.
- The OS loads:
  - program code;
  - static data.
- These elements are placed into the process address space.
- The process can then be prepared for execution.

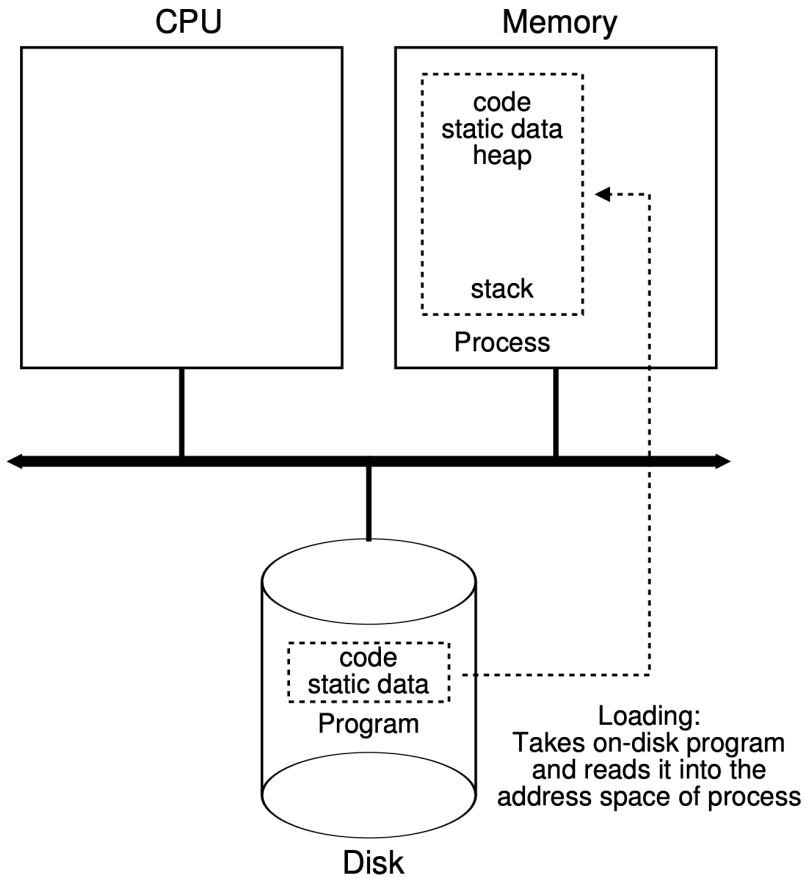


Figure 4.1: **Loading: From Program To Process**

Source: chapter page 4.

Process Creation

Before starting execution, the OS performs several steps:

1. Load code and static data into memory.
2. Allocate the run-time **stack**.
3. Initialize arguments such as `argc` and `argv`.
4. Allocate initial space for the **heap**.
5. Initialize I/O resources.
6. Transfer control to the program entry point.

UNIX Processes

A process normally starts with three open file descriptors: standard input, standard output, and standard error.

Loading: Eager vs. Lazy

- Early or simple operating systems may load the entire program before execution.

Eager Loading

Code and data are loaded into memory all at once before execution.

Lazy Loading

Modern systems can load pieces of code and data only when they are needed during execution.

Lazy loading is related to memory virtualization mechanisms such as paging and swapping.

Process States

A simplified process model contains three states:

- **Running**: executing instructions on a CPU.
- **Ready**: able to run, but not currently selected.
- **Blocked**: waiting for an event, such as I/O completion.

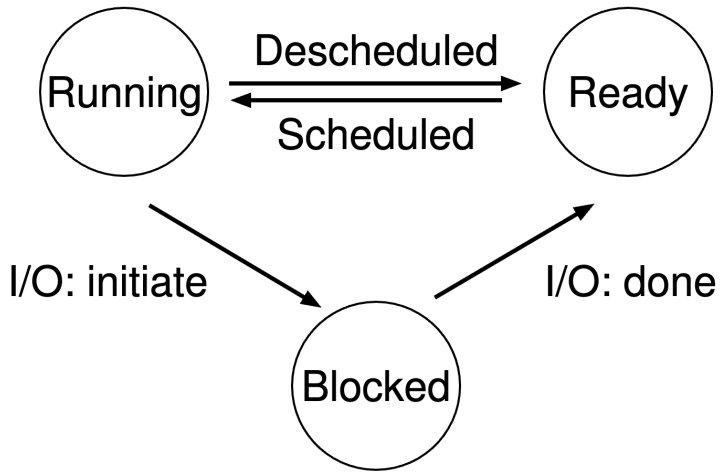


Figure 4.2: **Process: State Transitions**

Source: chapter page 6

Process State Transitions

- Ready $\rightarrow$ Running
  - the process is **scheduled**.
- Running $\rightarrow$ Ready
  - the process is **descheduled**.
- Running $\rightarrow$ Blocked
  - the process initiates an operation such as I/O.
- Blocked $\rightarrow$ Ready
  - the event or I/O operation completes.

Example: CPU-Only Processes

- Consider two processes that only use the CPU.
- One process runs while the other remains ready.
- When the first process finishes, the second can run.

Time	Process_0	Process_1	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	
4	Running	Ready	Process_0 now done
5	–	Running	
6	–	Running	
7	–	Running	
8	–	Running	Process_1 now done

Figure 4.3: **Tracing Process State: CPU Only**

Source: chapter page 6.

Example: CPU and I/O

- Process0 starts running.
- Process0 initiates I/O and becomes blocked.
- Process1 uses the CPU while Process0 waits.
- When I/O completes, Process0 becomes ready again.
- This improves CPU utilization.

Time	Process_0	Process_1	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	Process_0 initiates I/O
4	Blocked	Running	Process_0 is blocked,
5	Blocked	Running	so Process_1 runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	Process_1 now done
9	Running	–	
10	Running	–	Process_0 now done

Figure 4.4: **Tracing Process State: CPU and I/O**

Source: chapter page 7.

Scheduling Decisions

Even a simple CPU/I/O example requires OS decisions.

- Which ready process should run?
- Should the OS switch processes when one becomes blocked?
- Should a process run immediately after its I/O completes?
- Should the currently running process continue?

Scheduler

The OS component responsible for deciding which process should use the CPU.

Process Data Structures

The OS must keep information about every process.

- process state;
- process identifier (PID);
- memory information;
- register context;
- open files;
- parent process;
- current directory.

```
// the registers xv6 will save and restore
// to stop and subsequently restart a process
struct context {
  int eip;
  int esp;
  int ebx;
  int ecx;
  int edx;
  int esi;
  int edi;
  int ebp;
};

// the different states a process can be in
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };

// the information xv6 tracks about each process
// including its register context and state
struct proc {
  char *mem;               // Start of process memory
  uint sz;                 // Size of process memory
  char *kstack;            // Bottom of kernel stack
                           // for this process
  enum proc_state state;   // Process state
  int pid;                 // Process ID
  struct proc *parent;     // Parent process
  void *chan;              // If !zero, sleeping on chan
  int killed;              // If !zero, has been killed
  struct file *ofile[NOFILE]; // Open files
  struct inode *cwd;       // Current directory
  struct context context; // Switch here to run process
  struct trapframe *tf;    // Trap frame for the
                           // current interrupt
};
```

Figure 4.5: **The xv6 Proc Structure**

Source: chapter page 8.

Process List and PCB

- The OS maintains a **process list** or **task list**.
- It tracks:
  - ready processes;
  - running processes;
  - blocked processes.
- Information about an individual process is often stored in a **Process Control Block (PCB)**.
- The saved register context allows the OS to stop and later resume a process.

Context Switch

The OS saves the state of one process and restores the state of another process.

Additional Process States

Real operating systems may use more states than the simplified model.

- A process may have an initial state while it is being created.
- A process may remain temporarily after it finishes execution.

Zombie State

In UNIX-based systems, an exited process can remain in a zombie state until its parent obtains its return status, typically using `wait()`.

Chapter Summary

- A **process** is the OS abstraction of a running program.
- CPU virtualization allows many processes to share a small number of physical CPUs.
- A process is characterized by:
  - its address space;
  - CPU registers;
  - I/O state.
- Processes transition among states such as **running**, **ready**, and **blocked**.
- The OS tracks processes through structures such as the **process list** and **PCB**.
- Mechanisms and policies together enable CPU virtualization.

Process API

Process Creation and Control

Key Question

What interfaces should the operating system provide for **process creation and control**?

- UNIX provides a distinctive process creation API.
- Three fundamental system calls:
  - `fork()` – create a new process;
  - `exec()` – execute a different program;
  - `wait()` – wait for a child process to finish.

The `fork()` System Call

- *`fork()`* creates a new process.
- The original process is the **parent**.
- The newly created process is the **child**.
- The child is an almost exact copy of the parent.
- Both continue execution after *`fork()`*.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int main(int argc, char *argv[]) {
6    printf("hello (pid:%d)\n", (int) getpid());
7    int rc = fork();
8    if (rc < 0) {
9      // fork failed
10     fprintf(stderr, "fork failed\n");
11     exit(1);
12   } else if (rc == 0) {
13     // child (new process)
14     printf("child (pid:%d)\n", (int) getpid());
15   } else {
16     // parent goes down this path (main)
17     printf("parent of %d (pid:%d)\n",
18             rc, (int) getpid());
19   }
20   return 0;
21 }
22
```

Figure 5.1: **Calling `fork()` (`p1.c`)**

Source: chapter page 2.

Return Value

- Parent: child PID.
- Child: *`0`*.
- Failure: negative value.

INSTITUTO FEDERAL Paraíba

Parent and Child Execution

- Each process has its own:
  - address space;
  - registers;
  - program counter.
- Parent and child may execute in different orders.
- The scheduler determines which process runs first.
- Execution after `fork()` is therefore **non-deterministic**.

Process Identifier

Each process is identified by a unique **PID** (Process Identifier).

The `wait()` System Call

- A parent may need to wait for a child to finish.
- *`wait()`* blocks the parent.
- The parent becomes runnable again after the child terminates.
- This can make execution order predictable.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main(int argc, char *argv[]) {
7    printf("hello (pid:%d)\n", (int) getpid());
8    int rc = fork();
9    if (rc < 0) {          // fork failed; exit
10     fprintf(stderr, "fork failed\n");
11     exit(1);
12   } else if (rc == 0) { // child (new process)
13     printf("child (pid:%d)\n", (int) getpid());
14   } else {               // parent goes down this path
15     int rc_wait = wait(NULL);
16     printf("parent of %d (rc_wait:%d) (pid:%d)\n",
17            rc, rc_wait, (int) getpid());
18   }
19   return 0;
20 }
21
```

Figure 5.2: **Calling `fork()` And `wait()` (`p2.c`)**

Source: chapter page 3.

Execution Order

With *`wait()`*, the child completes before the parent continues past the system call.

The exec( ) System Call

- *exec( )* runs a different program.
- It replaces the current process image.
- Code and static data are replaced.
- Heap and stack are re-initialized.
- It does **not** create a new process.
- A successful *exec( )* does not return.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <sys/wait.h>
6
7  int main(int argc, char *argv[]) {
8    printf("hello (pid:%d)\n", (int) getpid());
9    int rc = fork();
10   if (rc < 0) {          // fork failed; exit
11     fprintf(stderr, "fork failed\n");
12     exit(1);
13   } else if (rc == 0) { // child (new process)
14     printf("child (pid:%d)\n", (int) getpid());
15     char *myargs[3];
16     myargs[0] = strdup("wc");   // program: "wc"
17     myargs[1] = strdup("p3.c"); // arg: input file
18     myargs[2] = NULL;           // mark end of array
19     execvp(myargs[0], myargs);  // runs word count
20     printf("this shouldn't print out");
21   } else {               // parent goes down this path
22     int rc_wait = wait(NULL);
23     printf("parent of %d (rc_wait:%d) (pid:%d)\n",
24            rc, rc_wait, (int) getpid());
25   }
26   return 0;
27 }
28
```

Figure 5.3: **Calling `fork()`, `wait()`, And `exec()` (`p3.c`)**

Source: chapter page 5.

Why Separate `fork()` and `exec()`?

- The separation is essential to the design of the UNIX shell.
- A shell typically:
  1. reads a command;
  2. calls *`fork()`*;
  3. modifies the child's environment if necessary;
  4. calls *`exec()`*;
  5. waits for completion with *`wait()`*.

Key Advantage

The shell can perform operations **between** *`fork()`* and *`exec()`*.

I/O Redirection

Example:

wc p3.c > newfile.txt

- The shell creates a child with *fork()*.
- The child closes standard output.
- It opens the destination file.
- The file becomes the new standard output.
- File descriptors remain open across *exec()*.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <fcntl.h>
6  #include <sys/stat.h>
7  #include <sys/wait.h>
8
9  int main(int argc, char *argv[]) {
10   int rc = fork();
11   if (rc < 0) {
12     // fork failed
13     fprintf(stderr, "fork failed\n");
14     exit(1);
15   } else if (rc == 0) {
16     // child: redirect standard output to a file
17     close(STDOUT_FILENO);
18     open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC,
19          S_IRWXU);
20     // now exec "wc"...
21     char *myargs[3];
22     myargs[0] = strdup("wc");   // program: wc
23     myargs[1] = strdup("p4.c"); // arg: file to count
24     myargs[2] = NULL;           // mark end of array
25     execvp(myargs[0], myargs);  // runs word count
26   } else {
27     // parent goes down this path (main)
28     int rc_wait = wait(NULL);
29   }
30   return 0;
31 }
```

Figure 5.4: **All Of The Above With Redirection (p4 . c)**

Source: chapter page 8.

Pipes

- UNIX pipes use the *pipe()* system call.
- The output of one process is connected to an in-kernel pipe.
- The input of another process is connected to the same pipe.
- This allows commands to be combined into processing chains.

Example

```
grep -o foo file | wc -l
```

The output of *grep* becomes the input of *wc*.

Process Control and Signals

- UNIX provides additional interfaces for process control.
- *kill()* sends signals to processes.
- Signals can stop, continue, interrupt, or terminate processes.

Common Shell Shortcuts

- *Ctrl+C* $\rightarrow$ *SIGINT*
- *Ctrl+Z* $\rightarrow$ *SIGTSTP*
- *fg* can resume a stopped job.

- Processes can use *signal()* to react to signals.

Users and Process Control

- UNIX systems support multiple users.
- Users generally control only their own processes.
- The operating system protects processes belonging to different users.
- The **superuser** (*root*) has broader privileges.
- Root can control processes belonging to other users.

Superuser

Administrative privileges should be used carefully because they provide extensive control over the system.

Useful Process Tools

- *ps* – displays running processes.
- *top* – displays processes and resource usage.
- *kill* – sends signals to processes.
- *killall* – another interface for sending signals.

Good Practice

Use UNIX *man* pages to study system calls, return values, options, and error conditions.

Process API: Summary

- *fork()* creates a new child process.
- *wait()* allows a parent to wait for a child.
- *exec()* replaces the current program with another.
- UNIX shells combine these mechanisms to execute commands.
- The separation of *fork()* and *exec()* enables redirection and pipes.
- Signals provide mechanisms for process control.
- Users and permissions determine which processes can be controlled.

Direct Execution

CPU Virtualization: The Main Challenge

Goal

The operating system must virtualize the CPU efficiently while retaining control over the system.

- The CPU is shared among multiple processes through **time sharing**.
- Two central challenges:
  - **Performance**: minimize virtualization overhead.
  - **Control**: prevent processes from taking over the system.
- Hardware and operating-system support are both required.

Basic Technique (Mechanism): Limited Direct Execution

- The basic idea is to run programs **directly on the CPU**.
- Direct execution provides high performance.
- Before execution, the OS:
  - creates a process entry;
  - allocates memory;
  - loads the program;
  - prepares the stack;
  - starts execution at the program entry point.

Problem

How can the OS run a program directly while still maintaining control over the CPU?

Direct Execution Without Limits

- The OS starts the program.
- The program executes directly on the processor.
- When it finishes, control returns to the OS.

Two problems emerge:

1. How to restrict privileged operations?
2. How to regain control of the CPU?

OS	Program
Create entry for process list	
Allocate memory for program	
Load program into memory	
Set up stack with argc/argv	
Clear registers	
Execute **call** main()	
	Run main()
	Execute **return** from main
Free memory of process	
Remove from process list	

Figure 6.1: **Direct Execution Protocol (Without Limits)**

Source: chapter page 2.

Problem #1: Restricted Operations

The Crux

A process must perform I/O and other restricted operations without gaining complete control over the system.

- Direct execution alone would allow applications to perform operations that should be controlled by the OS.
- The processor therefore supports different execution modes:
  - **User mode**: restricted execution.
  - **Kernel mode**: privileged execution.
- Privileged operations are performed by the operating system.

System Calls and Protected Control Transfer

- User programs request OS services through **system calls**.
- A system call uses a special **trap instruction**.
- The trap:
  1. saves the required processor state;
  2. switches execution to kernel mode;
  3. transfers control to a predefined OS handler.
- The OS performs the requested operation.
- A **return-from-trap** instruction restores execution in user mode.

Key Idea

System calls provide controlled access to privileged OS services.

Why System Calls Look Like Procedure Calls

- Calls such as *`open()`* and *`read()`* appear to be ordinary C procedure calls.
- The C library prepares:
  - system-call arguments;
  - the system-call number.
- Library code then executes the hardware-specific **trap instruction**.
- After returning from the kernel, the library processes the return value and returns control to the application.

The Trap Table

- User programs cannot choose arbitrary kernel addresses as trap destinations.
- During boot, the OS initializes a **trap table**.
- The table defines handlers for events such as:
  - system calls;
  - hardware interrupts;
  - exceptional conditions.
- Installing the trap table is a **privileged operation**.

Protection

User code requests a service through a system-call number, rather than specifying an arbitrary kernel address.

Limited Direct Execution Protocol

OS @ boot (kernel mode)	Hardware	
initialize trap table		
	remember address of... syscall handler	

OS @ run (kernel mode)	Hardware	Program (user mode)
Create entry for process list		
Allocate memory for program		
Load program into memory		
Setup user stack with argv		
Fill kernel stack with reg/PC		
return-from-trap		
	restore regs	
	(from kernel stack)	
	move to user mode	
	jump to main	
		Run main()
		...
		Call system call
		trap into OS
	save regs	
	(to kernel stack)	
	move to kernel mode	
	jump to trap handler	
Handle trap		
Do work of syscall		
return-from-trap		
	restore regs	
	(from kernel stack)	
	move to user mode	
	jump to PC after trap	
		...
		return from main
		trap (via `exit()`)
Free memory of process		
Remove from process list		

Figure 6.2: **Limited Direct Execution Protocol**

Source: chapter page 5.

Problem #2: Switching Between Processes

The Crux

How can the operating system regain control of the CPU while a process is running?

- If a user process is executing, the OS itself is not executing.
- The OS therefore needs a mechanism to regain control.
- Two approaches are discussed:
  - cooperative;
  - non-cooperative.

Cooperative Approach

- The OS waits for the process to voluntarily transfer control.
- Control can return to the OS through:
  - system calls;
  - an explicit `yield()` system call;
  - traps caused by illegal operations.

Limitation

A process running forever without making a system call could monopolize the CPU.

Non-Cooperative Approach: Timer Interrupt

- Hardware provides a programmable **timer interrupt**.
- The timer periodically interrupts the running process.
- When the interrupt occurs:
  1. the running process is halted;
  2. its state is saved;
  3. the CPU switches to kernel mode;
  4. the OS interrupt handler executes.
- The OS has regained control of the CPU.

Key Mechanism

The timer interrupt prevents a process from monopolizing the CPU.

Saving and Restoring Context

- After regaining control, the OS scheduler decides whether:
  - to continue executing the current process; or
  - to switch to another process.
- A **context switch** saves the state of one process and restores the state of another.
- Relevant state includes:
  - general-purpose registers;
  - program counter;
  - kernel stack pointer.

Timer Interrupt and Context Switch

- Process A is executing.
- A timer interrupt occurs.
- Hardware saves A's user state.
- The OS enters the interrupt handler.
- The OS switches from A to B.
- B resumes execution.

OS @ boot (kernel mode)	Hardware
initialize trap table	
	remember addresses of...
	syscall handler
	timer handler
start interrupt timer	
	start timer
	interrupt CPU in X ms

OS @ run (kernel mode)	Hardware	Program (user mode)
		Process A
		...
	timer interrupt	
	save regs(A) → k-stack(A)	
	move to kernel mode	
	jump to trap handler	
Handle the trap		
Call `switch()` routine		
save regs(A) → proc_t(A)		
restore regs(B) ← proc_t(B)		
switch to k-stack(B)		
return-from-trap (into B)		
	restore regs(B) ← k-stack(B)	
	move to user mode	
	jump to B's PC	
		Process B
		...

Figure 6.3: **Limited Direct Execution Protocol (Timer Interrupt)**

Source: chapter page 11.

Two Types of State Saving

1. **Hardware save/restore**
   - occurs when the timer interrupt happens;
   - saves the user registers;
   - uses the process kernel stack.

2. **Software save/restore**
   - occurs during the context switch;
   - performed explicitly by the OS;
   - saves and restores kernel context.

Example: xv6 Context Switch

- Save registers of the old process.
- Restore registers of the new process.
- Switch the kernel stack.
- Return into the new execution context.

```
1  # void swtch(struct context *old, struct context *new);
2  #
3  # Save current register context in old
4  # and then load register context from new.
5  .globl swtch
6  swtch:
7    # Save old registers
8    movl 4(%esp), %eax  # put old ptr into eax
9    popl 0(%eax)        # save the old IP
10   movl %esp, 4(%eax)  # and stack
11   movl %ebx, 8(%eax)  # and other registers
12   movl %ecx, 12(%eax)
13   movl %edx, 16(%eax)
14   movl %esi, 20(%eax)
15   movl %edi, 24(%eax)
16   movl %ebp, 28(%eax)
17
18   # Load new registers
19   movl 4(%esp), %eax  # put new ptr into eax
20   movl 28(%eax), %ebp # restore other registers
21   movl 24(%eax), %edi
22   movl 20(%eax), %esi
23   movl 16(%eax), %edx
24   movl 12(%eax), %ecx
25   movl 8(%eax), %ebx
26   movl 4(%eax), %esp  # stack is switched here
27   pushl 0(%eax)       # return addr put in place
28   ret                 # finally return into new ctxt
```

Figure 6.4: **The xv6 Context Switch Code**

Source: chapter page 12.

What About Concurrency?

- What happens if a timer interrupt occurs during a system call?
- What happens if another interrupt arrives while one is being handled?

The chapter briefly introduces two approaches:

- temporarily disabling interrupts during interrupt processing;
- using locking mechanisms to protect kernel data structures.

Next Step

Detailed treatment of these problems belongs to the study of **concurrency**.

Limited Direct Execution: Summary

- Programs execute directly on the CPU for high performance.
- **User mode** restricts application privileges.
- **Kernel mode** allows privileged operations.
- **System calls** use traps to enter the kernel safely.
- A **trap table** determines valid kernel entry points.
- A **timer interrupt** allows the OS to regain CPU control.
- A **context switch** changes the running process.

Central Idea

Limited direct execution combines efficient execution with operating-system control.

Performance Measurement

The chapter suggests measuring two fundamental operating-system costs:

- System-call cost
  - repeatedly execute a simple system call;
  - measure total execution time;
  - estimate the average cost per call.
- Context-switch cost
  - use two processes communicating through UNIX pipes;
  - force repeated switches between the processes;
  - measure the communication cycle.

Tools Mentioned in the Chapter

gettimeofday(), *rdtsc*, *sched_setaffinity()*, and *lmbench*.

Direct Execution: Summary

- **Limited Direct Execution** combines performance and OS control.
- **User/kernel modes** restrict privileged operations.
- **System calls** use traps to safely enter the kernel.
- **Timer interrupts** allow the OS to regain CPU control.
- **Context switches** enable execution to move between processes.

Next Step

With these mechanisms in place, the remaining question is:

Which process should run at a given time?

This is the responsibility of the **CPU scheduler**.

CPU Scheduling

From Mechanisms to Scheduling

- We now understand the low-level mechanisms for running processes:
  - timer interrupts;
  - context switching;
  - switching between processes.
- The next question concerns **policy**:
  - Which process should run?
  - When should it run?
  - For how long?

The Crux

How should we develop a basic framework for thinking about **scheduling policies**?

Workload Assumptions

To develop basic scheduling policies, we initially assume:

1. Each job runs for the same amount of time.
2. All jobs arrive at the same time.
3. Once started, each job runs to completion.
4. Jobs use only the CPU and perform no I/O.
5. The run-time of each job is known.

These assumptions are intentionally unrealistic and will be **relaxed progressively**.

Scheduling Metrics

The main performance metric initially considered is **turnaround time**:

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$$

With all jobs arriving at $t = 0$:

$$T_{\text{turnaround}} = T_{\text{completion}}$$

Performance vs. Fairness

Scheduling policies may improve performance at the cost of fairness, or improve fairness at the cost of performance.

First In, First Out (FIFO)

- Also called **First Come, First Served (FCFS)**.
- Simple and easy to implement.
- Jobs execute in arrival order.

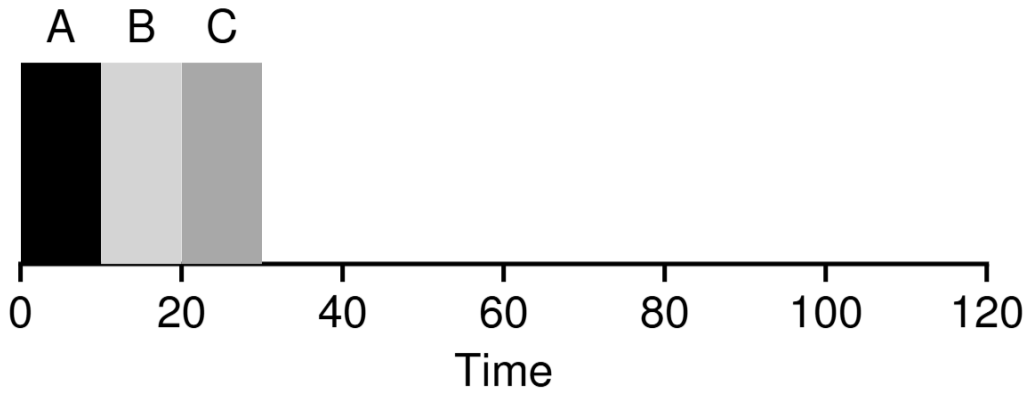


Figure 7.1: **FIFO Simple Example**

Source: chapter page 3.

Example:

$$A = B = C = 10$$

$$\overline{T}_{turnaround} = \frac{10 + 20 + 30}{3} = 20$$

The Convoy Effect

Relax assumption 1: jobs may have **different lengths**.

Example:

$$A = 100, \qquad B = C = 10$$

Under FIFO:

$$\overline{T}_{turnaround} = \frac{100 + 110 + 120}{3} = 110$$

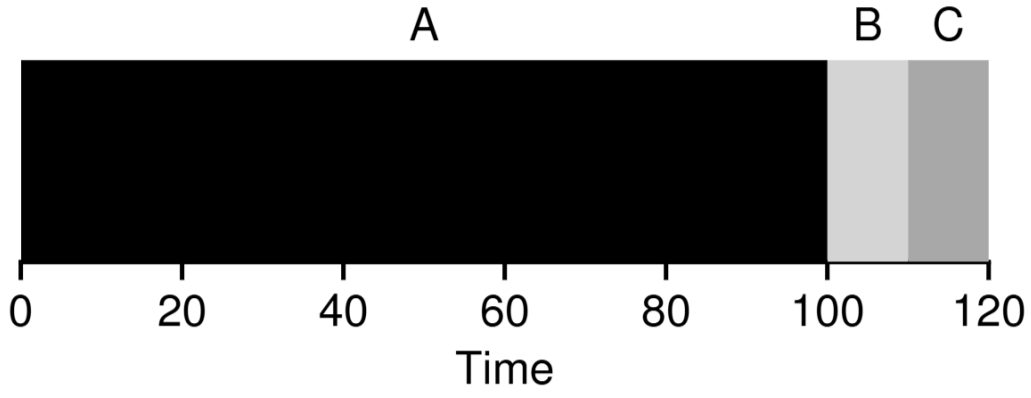


Figure 7.2: **Why FIFO Is Not That Great**

Source: chapter page 3.

Convoy Effect

Short jobs wait behind a long-running job.

Shortest Job First (SJF)

Shortest Job First (SJF) executes jobs in increasing order of run time.

For:

$$A = 100, \qquad B = C = 10$$

SJF executes:

$$B \rightarrow C \rightarrow A$$

and obtains:

$$\overline{T}_{turnaround} = \frac{10 + 20 + 120}{3} = 50$$

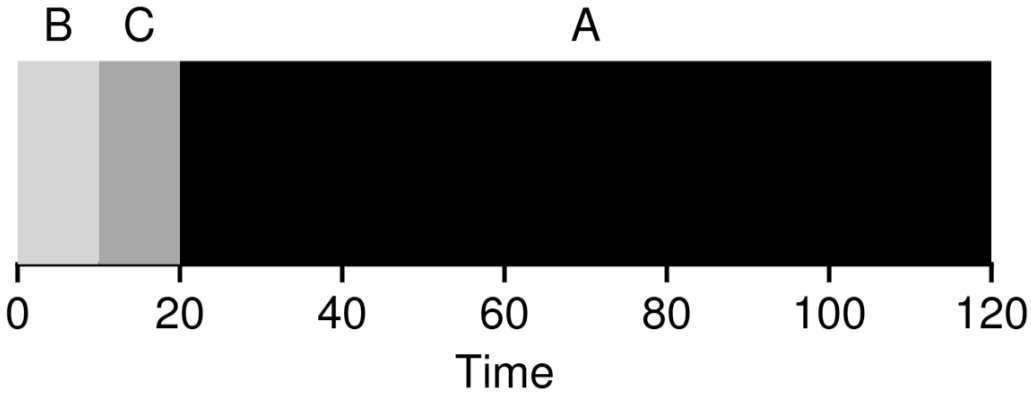


Figure 7.3: **SJF Simple Example**

Source: chapter page 4.

SJF and Late Arrivals

Now relax assumption 2: jobs may arrive at **different times**.

Example:

- A arrives at $t = 0$, requiring 100 s.
- B and C arrive at $t = 10$, requiring 10 s each.

Because SJF is non-preemptive, B and C must still wait for A.

$$\overline{T}_{turnaround} = 103.33$$

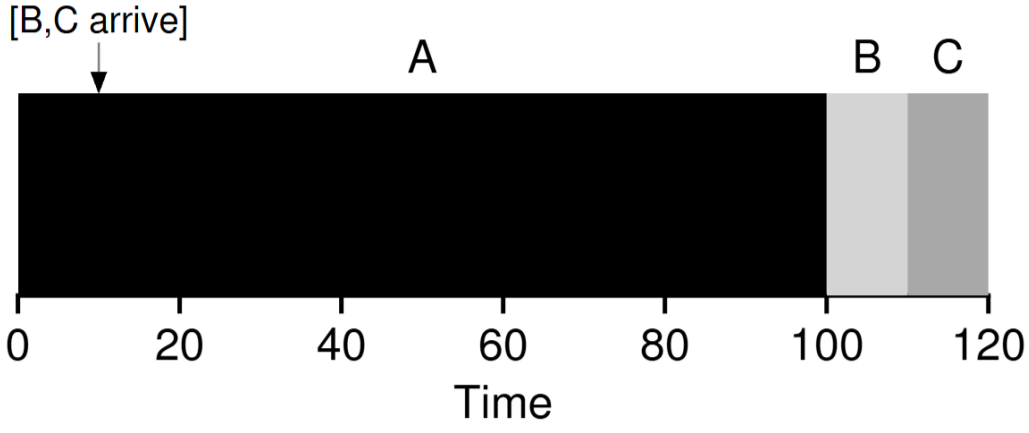


Figure 7.4: **SJF With Late Arrivals From B and C**

Source: chapter page 5.

Shortest Time-to-Completion First (STCF)

Relax assumption 3: a running job may be **preempted**.

STCF is the preemptive version of SJF.

- When a new job arrives, compare the remaining execution times.
- Run the job with the least time remaining.
- A running process may therefore be preempted.

For the previous example:

$$\overline{T}_{turnaround} = 50$$

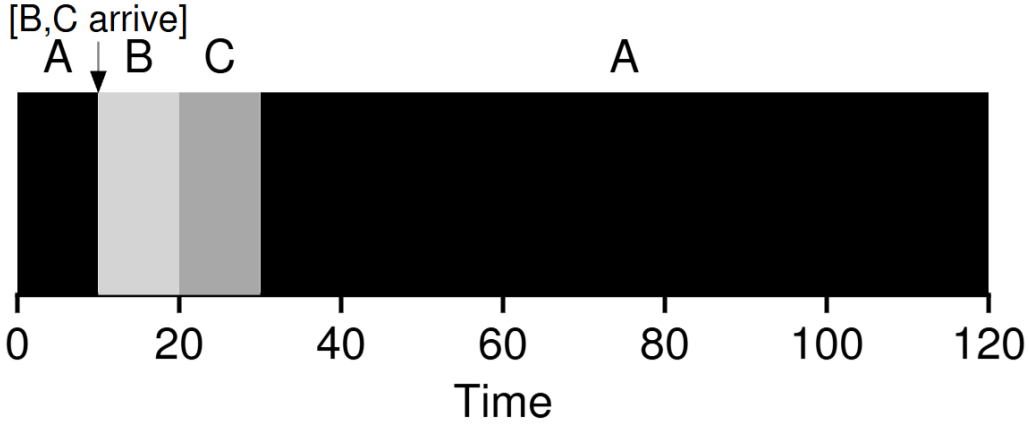


Figure 7.5: **STCF Simple Example**

Source: chapter page 6.

A New Metric: Response Time

Interactive systems require another metric: **response time**.

$$T_{\text{response}} = T_{\text{firstrun}} - T_{\text{arrival}}$$

- Turnaround time focuses on **completion**.
- Response time focuses on how quickly a job **starts receiving CPU time**.
- SJF/STCF may provide excellent turnaround time but poor interactive response.

SJF vs. Round Robin

SJF

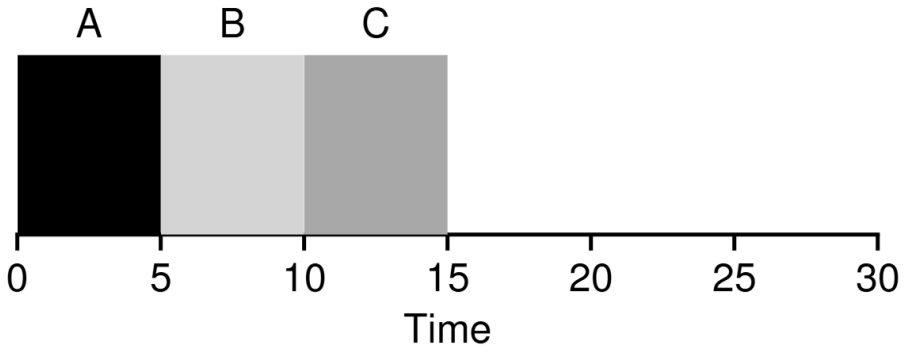


Figure 7.6: **SJF Again (Bad for Response Time)**

Source: chapter page 7.

$$\overline{T}_{response} = \frac{0 + 5 + 10}{3} = 5$$

Round Robin

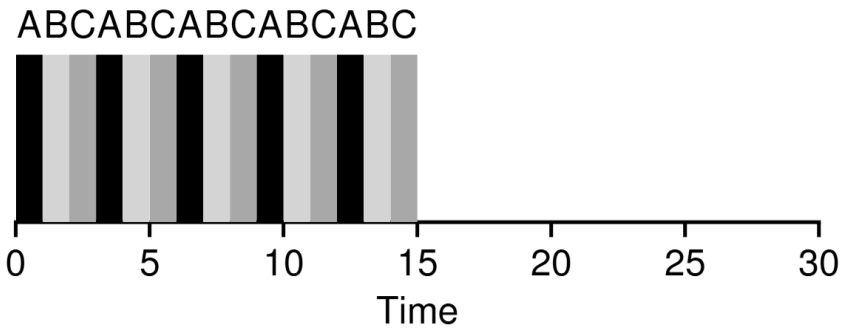


Figure 7.7: **Round Robin (Good For Response Time)**

Source: chapter page 7.

$$\overline{T}_{response} = \frac{0 + 1 + 2}{3} = 1$$

Round Robin (RR)

Round Robin introduces **time slicing**.

- Each job runs for a fixed **time slice** or **scheduling quantum**.
- After the quantum expires, the scheduler switches to the next job in the run queue.
- The process repeats until all jobs finish.

Time-Slice Trade-off

A shorter time slice improves response time, but increases the relative cost of context switching.

- Example: 10 ms quantum + 1 ms context switch $\Rightarrow$ roughly 10% switching overhead.
- Increasing the quantum amortizes this fixed cost.

Turnaround Time vs. Response Time

The scheduling policies studied so far optimize different goals.

Policy	Turnaround	Response
FIFO	workload-dependent	poor with convoy effect
SJF	very good	potentially poor
STCF	very good	potentially poor
RR	poor	very good

Fundamental Trade-off

Policies that divide CPU time fairly among active processes tend to improve response time, but usually increase turnaround time.

Incorporating I/O

Relax assumption 4: processes may perform **I/O**.

- A process issuing I/O becomes blocked.
- The scheduler can run another process while the I/O proceeds.
- When the I/O completes, the blocked process becomes ready again.

Example:

- Job A: five 10-ms CPU bursts separated by I/O.
- Job B: one continuous 50-ms CPU burst.

Overlapping CPU and I/O

Poor Resource Use

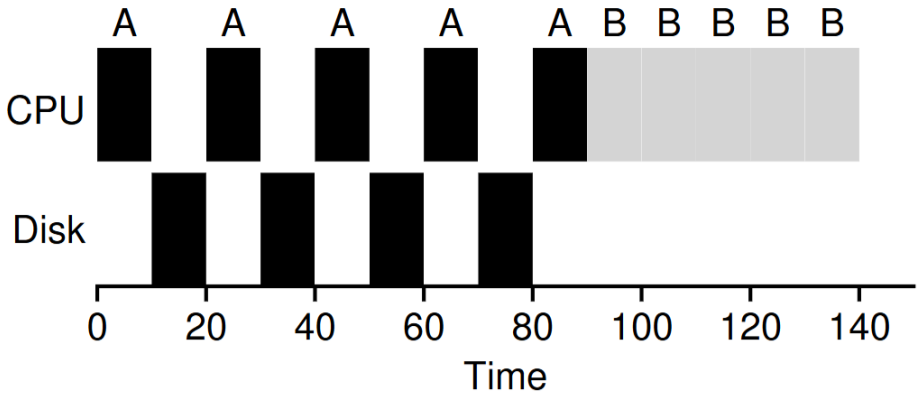


Figure 7.8: **Poor Use Of Resources**

Source: chapter page 10.

CPU/I/O Overlap

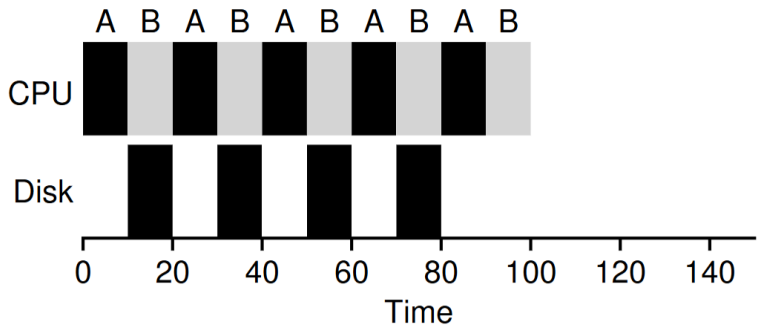


Figure 7.9: **Overlap Allows Better Use Of Resources**

Source: chapter page 10.

Key Idea

Treating each CPU burst as a separate job enables CPU execution to overlap with another process's I/O, improving system utilization.

No More Oracle

One assumption still remains:

The scheduler knows the run-time of each job.

In a general-purpose operating system, this information is normally **not known in advance**.

The remaining challenge is therefore:

- obtain SJF/STCF-like turnaround behavior;
- preserve good response time;
- operate without knowing future job lengths.

Next Step

Use the **recent past to predict the future**: the Multi-Level Feedback Queue (MLFQ).

CPU Scheduling: Summary

- Scheduling policy determines **which process runs**.
- **FIFO** is simple but may suffer from the convoy effect.
- **SJF/STCF** optimize turnaround time.
- **Round Robin** improves response time through time slicing.
- Response time and turnaround time represent an inherent scheduling trade-off.
- CPU and I/O can be overlapped to improve utilization.
- Real systems must schedule without knowing future job lengths.

Next: Multi-Level Feedback Queue (MLFQ)

Multi-Level Feedback Queue

Multi-Level Feedback Queue (MLFQ)

- MLFQ addresses two main scheduling goals:
  - Minimize **turnaround time**.
  - Minimize **response time** for interactive jobs.
- Challenge:
  - The OS usually does not know how long a job will run.
  - SJF/STCF require such knowledge.
  - Round Robin improves responsiveness but may hurt turnaround time.

The Crux

How can a scheduler achieve good response and turnaround times without a priori knowledge of job length?

Learning From History

- MLFQ uses the **observed behavior** of jobs to determine priorities.
- It uses past behavior to predict future behavior.
- Interactive jobs tend to:
  - run for short periods;
  - frequently relinquish the CPU for I/O.
- CPU-bound jobs tend to use the CPU for long periods.

MLFQ: Basic Structure

- MLFQ contains multiple queues with different priority levels.
- A ready job belongs to exactly one queue.
- Higher-priority jobs run before lower-priority jobs.
- Jobs at the same priority are scheduled using Round Robin.

Basic Rules

Rule 1: If Priority(A) $>$ Priority(B), A runs.

Rule 2: If Priority(A) = Priority(B), A and B run in RR.

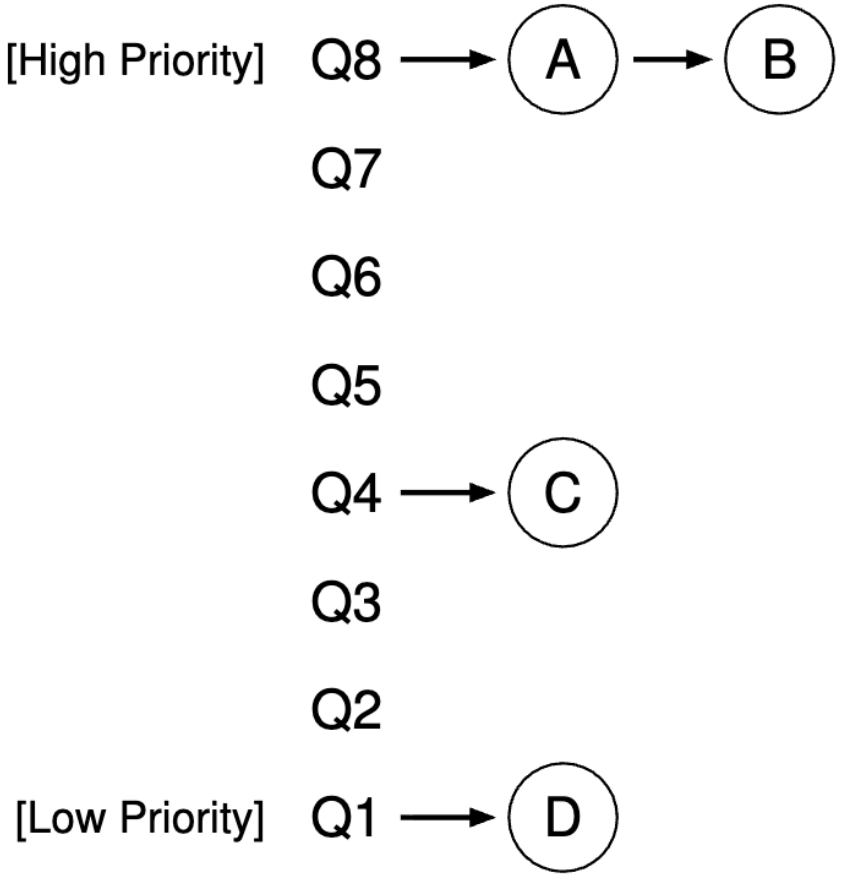


Figure 8.1: **MLFQ Example**

Source: chapter page 3.

Changing Priority: First Attempt

- A job's **allotment** is the amount of CPU time it may use at a priority level before being demoted.
- Initially, assume the allotment equals one time slice.

Initial Priority Rules

Rule 3: A new job enters at the highest priority.

Rule 4a: If a job uses its entire allotment, its priority is reduced.

Rule 4b: If a job gives up the CPU before its allotment expires, it stays at the same priority.

Example 1: A Long-Running Job

- Consider three priority queues.
- Time slice: 10 ms.
- The job starts in the highest-priority queue.
- After consuming each allotment, it moves down one queue.
- Eventually, it remains in the lowest-priority queue.

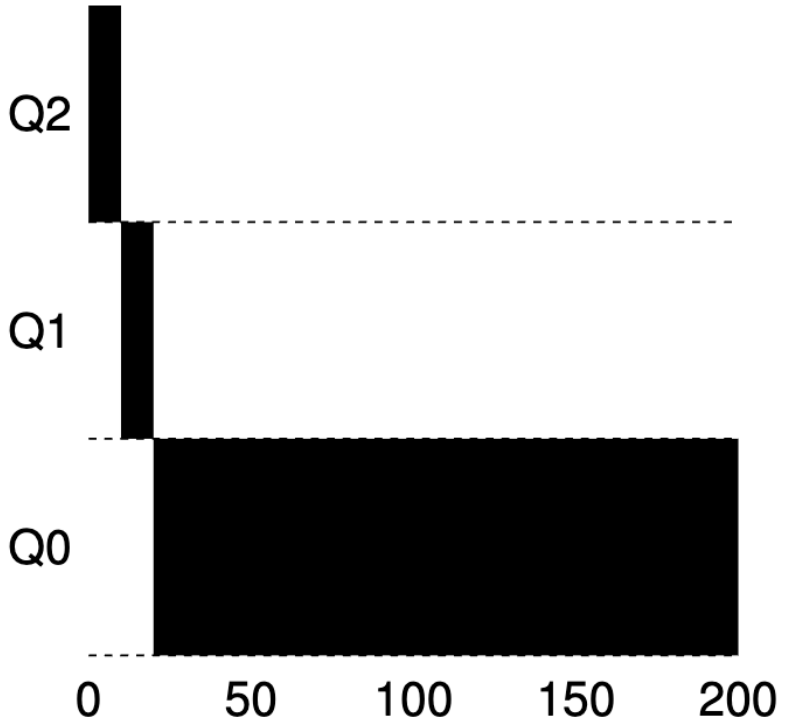


Figure 8.2: Long-running Job Over Time

Source: chapter page 4.

Example 2: A Short Job Arrives

- Job A is a long-running CPU-intensive job.
- Job B is a newly-arrived short job.
- B enters at the highest priority.
- Because B is short, it completes before reaching the lowest queue.
- MLFQ therefore approximates SJF without knowing job lengths.

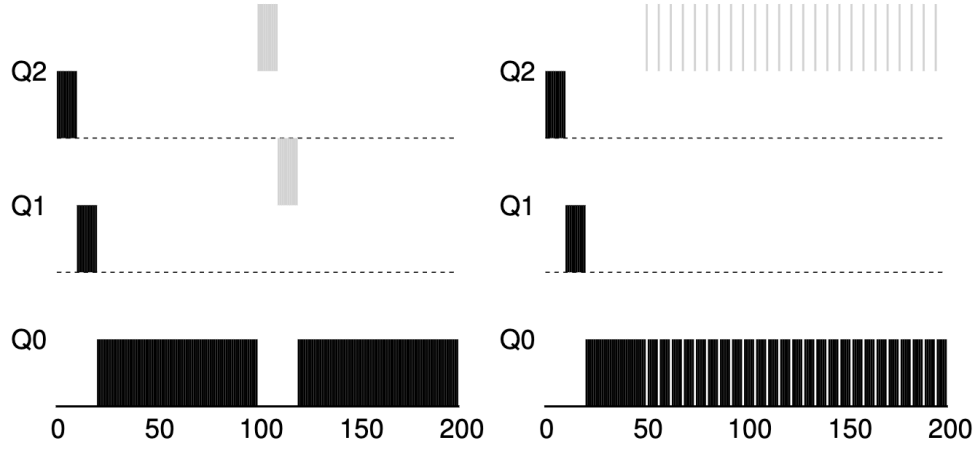


Figure 8.3: **Along Came An Interactive Job: Two Examples**

Source: chapter page 5.

Example 3: Interactive Jobs and I/O

- Interactive jobs often relinquish the CPU for I/O.
- Under Rule 4b, such jobs remain at high priority.
- This allows interactive jobs to receive fast CPU service.

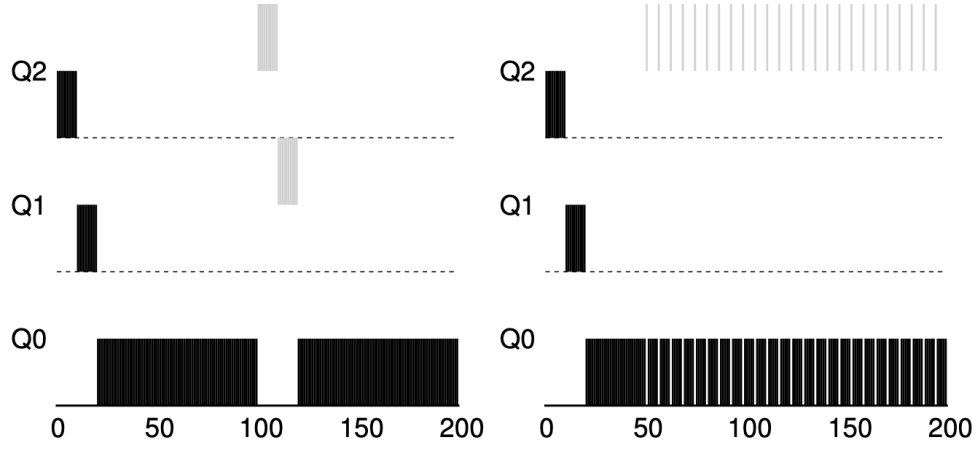


Figure 8.3: **Along Came An Interactive Job: Two Examples**

Source: chapter page 5.

Problems With the Initial MLFQ

1. **Starvation**
   - Too many interactive jobs may consume all CPU time.
   - Long-running jobs may never execute.
2. **Gaming the Scheduler**
   - A job can issue I/O just before its allotment expires.
   - It can remain at high priority and obtain an unfair CPU share.
3. **Changing Behavior**
   - A CPU-bound process may later become interactive.
   - The current rules do not adapt well to this change.

Attempt #2: Priority Boost

Rule 5

After some time period S, move all jobs in the system to the topmost queue.

- Prevents starvation of long-running jobs.
- Allows jobs whose behavior changed to regain high priority.
- However, choosing a proper value for S is difficult.

Priority Boost in Action

- Without priority boosts:
  - a long-running job may starve.
- With periodic boosts:
  - the long-running job periodically receives CPU time.

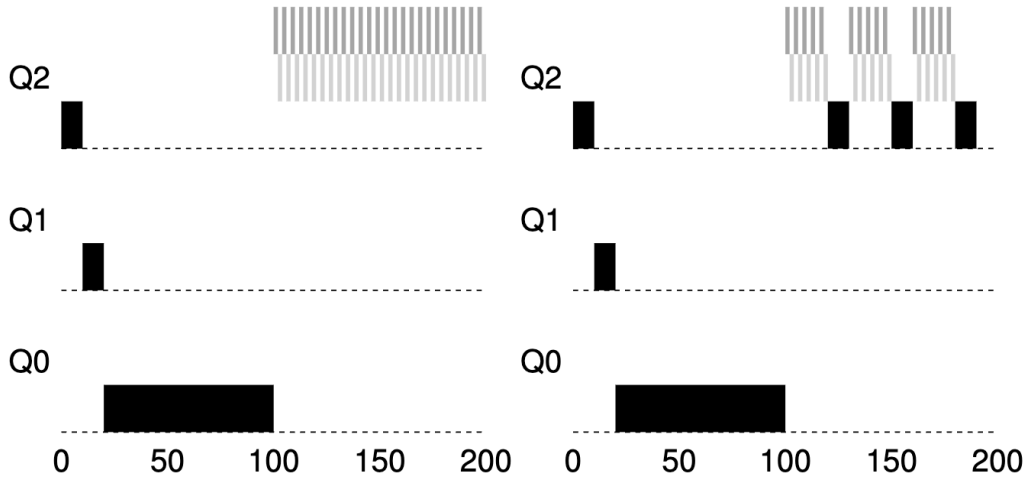


Figure 8.4: **Without (Left) and With (Right) Priority Boost**

Source: chapter page 7.

Attempt #3: Better Accounting

- Rules 4a and 4b allow processes to game the scheduler.
- The solution is to track the **total CPU time used at each level**.
- Giving up the CPU for I/O no longer resets the allotment.

Revised Rule 4

Once a job uses up its time allotment at a given level, regardless of how many times it has given up the CPU, its priority is reduced.

Preventing Scheduler Gaming

- Old rules:
  - I/O could be used to remain at high priority.
- Better accounting:
  - CPU usage is accumulated.
  - The process eventually moves down the queues.

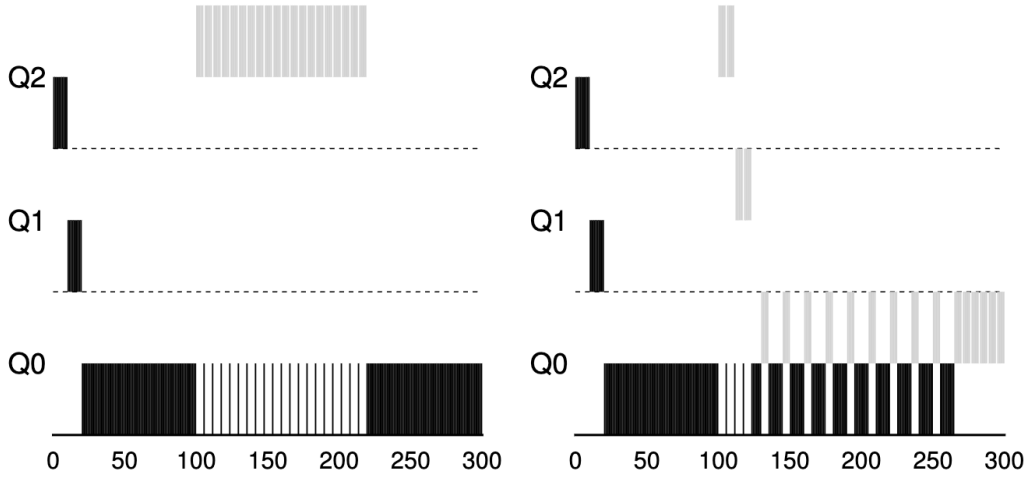


Figure 8.5: **Without (Left) and With (Right) Gaming Tolerance**

Source: chapter page 8.

Tuning MLFQ

- Important parameters include:
  - number of queues;
  - time slice per queue;
  - allotment per priority level;
  - frequency of priority boosts.
- High-priority queues usually use **short time slices**.
- Low-priority queues usually use **longer time slices**.
- Interactive jobs benefit from quick switching.
- CPU-bound jobs benefit from longer uninterrupted execution.

Lower Priority, Longer Quanta

- Example:
  - highest queue: 10-ms time slice;
  - middle queue: 20-ms time slice;
  - lowest queue: 40-ms time slice.

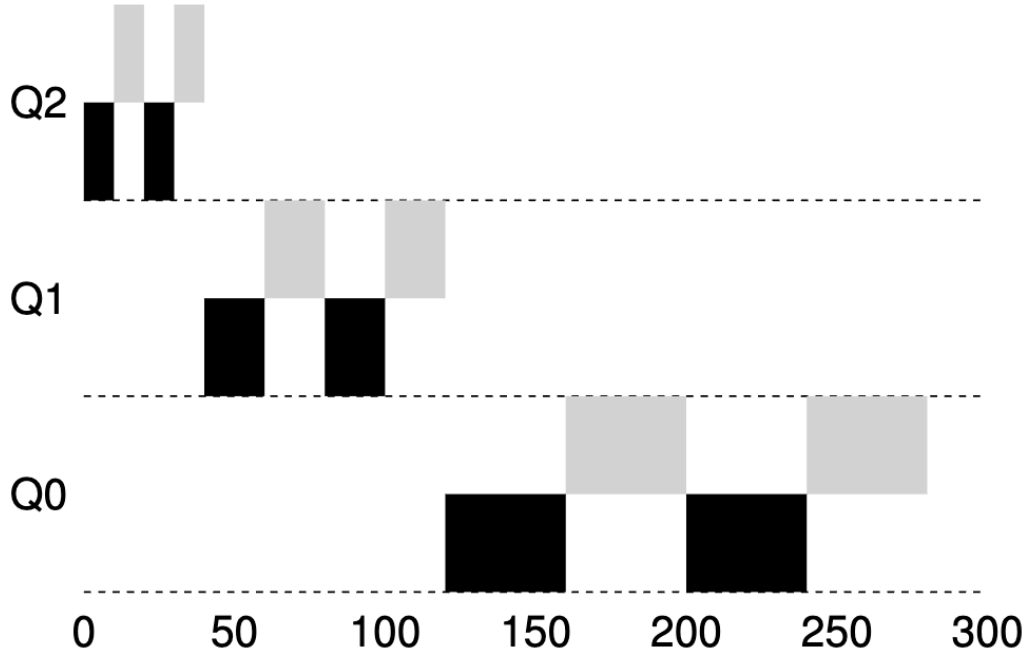


Figure 8.6: **Lower Priority, Longer Quanta**

Source: chapter page 9.

MLFQ in Real Systems

- Different systems implement variations of MLFQ.
- Solaris Time-Sharing (TS):
  - configurable priority tables;
  - 60 queues in the default configuration;
  - increasing time-slice lengths at lower priorities;
  - periodic priority boosts.
- FreeBSD uses mathematical formulas based on CPU usage.
- Some systems reserve the highest priority levels for OS work.
- Utilities such as *`nice`* allow users to provide scheduling advice.

MLFQ: Final Rules

1. If Priority(A) $>$ Priority(B), A runs.
2. If Priority(A) = Priority(B), A and B run in Round Robin.
3. New jobs enter the system at the highest priority.
4. Once a job uses its time allotment at a level, its priority is reduced.
5. After some period *S*, all jobs are moved to the highest-priority queue.

Summary

- MLFQ uses multiple priority queues and feedback from job behavior.
- It does not require a priori knowledge of job length.
- Short and interactive jobs tend to remain at high priority.
- CPU-intensive jobs gradually move to lower-priority queues.
- Priority boosts prevent starvation and adapt to behavioral changes.
- Better accounting prevents jobs from gaming the scheduler.
- MLFQ combines good responsiveness with progress for long-running jobs.

Lottery Scheduling

Proportional-Share Scheduling

- Proportional-share schedulers focus on **fair CPU allocation**.
- Instead of optimizing turnaround or response time, the goal is to:
  - guarantee that each job receives a certain percentage of CPU time.
- Lottery scheduling is an early and influential approach.

The Crux

How can we design a scheduler to share the CPU proportionally?

Lottery Scheduling

- Lottery scheduling uses **randomness** to select the next process.
- Each process receives a number of **tickets**.
- Tickets represent a process's share of the CPU.
- A lottery is held periodically, typically once per time slice.
- The process holding the winning ticket runs next.

Tickets Represent CPU Share

- Suppose:
  - Process A has 75 tickets.
  - Process B has 25 tickets.
- Desired CPU allocation:

$$A = 75\%, \qquad B = 25\%$$

- The scheduler randomly selects one ticket among all available tickets.
- Allocation is **probabilistic**, not deterministic.
- Over longer execution periods, the obtained proportions tend to approach the desired shares.

Why Randomness?

- Randomized scheduling has several useful properties:
  - avoids some pathological corner cases;
  - requires little per-process state;
  - can make scheduling decisions quickly.
- Lottery scheduling only needs:
  - the number of tickets per process;
  - the total number of tickets;
  - a random number generator.

Ticket Mechanisms

Ticket Currency Users can distribute tickets among their own jobs using a local currency, which is converted into a global ticket value.

Ticket Transfer A process can temporarily transfer its tickets to another process.

Ticket Inflation A process can temporarily increase or decrease its number of tickets in environments where processes trust one another.

Lottery Scheduling: Implementation

- Keep runnable processes in a data structure such as a list.
- Generate a random winning ticket.
- Traverse the process list while accumulating ticket counts.
- The process whose accumulated count exceeds the winning ticket is selected.

```
1  // counter: used to track if we've found the winner yet
2  int counter = 0;
3
4  // winner: call some random number generator to
5  //         get a value >= 0 and <= (totaltickets - 1)
6  int winner = getrandom(0, totaltickets);
7
8  // current: use this to walk through the list of jobs
9  node_t *current = head;
10 while (current) {
11     counter = counter + current->tickets;
12     if (counter > winner)
13         break; // found the winner
14     current = current->next;
15 }
16 // 'current' is the winner: schedule it...
```

Figure 9.1: **Lottery Scheduling Decision Code**

Source: chapter page 4.

Lottery Scheduling Example

- Consider three jobs:
  - A: 100 tickets;
  - B: 50 tickets;
  - C: 250 tickets.
- Total number of tickets:

$$100 + 50 + 250 = 400$$

- If the winning ticket is 300:
  - A covers tickets up to 100;
  - A+B cover tickets up to 150;
  - A+B+C cover tickets up to 400.
- Therefore, process C wins the lottery.

Lottery Scheduling Fairness

- Consider two jobs with:
  - equal ticket allocations;
  - equal runtime R.
- Fairness is defined as:

$$F = \frac{\text{completion time of first job}}{\text{completion time of second job}}$$

- A perfectly fair result gives:

$$F = 1$$

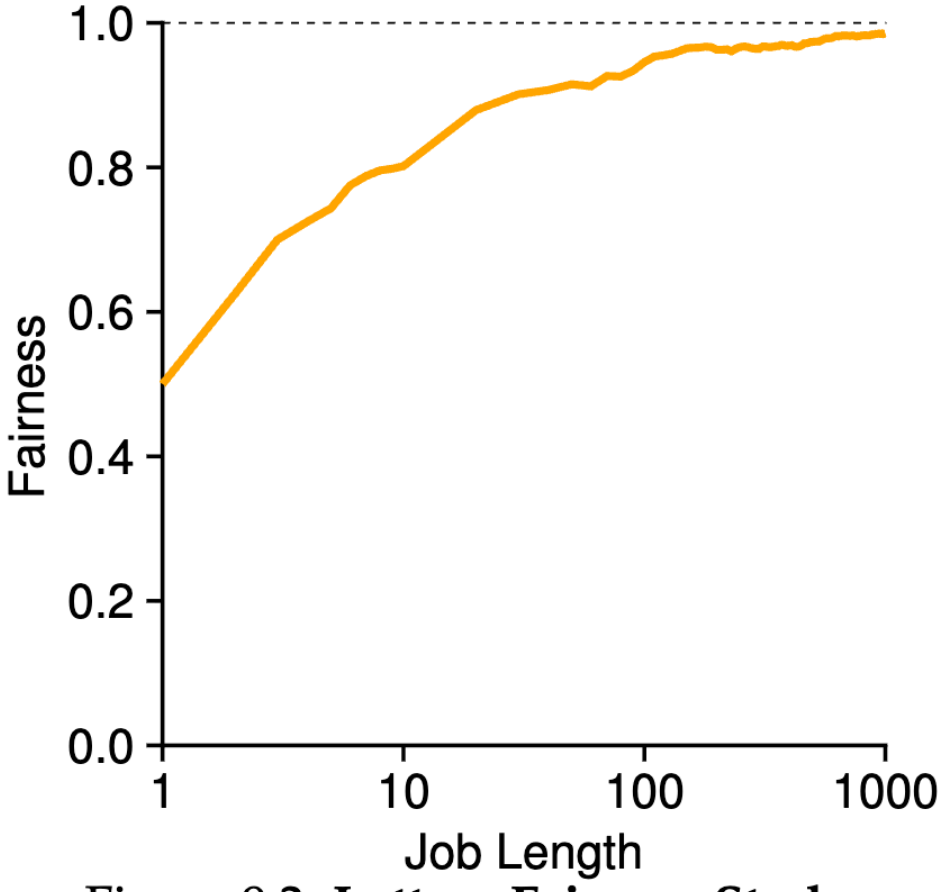


Figure 9.2: Lottery Fairness Study

Source: chapter page 5.

Fairness Over Time

- For short jobs, lottery scheduling may produce significant unfairness.
- Randomness can cause one job to finish substantially earlier than another.
- As job length increases:
  - more lotteries are performed;
  - average fairness approaches the desired result.
- Thus, lottery scheduling achieves proportional fairness **probabilistically over time**.

How Should Tickets Be Assigned?

- The behavior of lottery scheduling strongly depends on ticket allocation.
- One possibility is to give users a fixed number of tickets.
- Users may then distribute their tickets among their jobs.
- However, the general **ticket-assignment problem** remains open.

Stride Scheduling

- Stride scheduling provides a **deterministic** approach to proportional-share scheduling.
- Each process has:
  - a number of tickets;
  - a **stride**;
  - a **pass value**.
- Stride is inversely proportional to the number of tickets:

$$stride_i = \frac{\text{large constant}}{tickets_i}$$

- Processes with more tickets therefore have smaller strides.

Stride Scheduling: Basic Operation

1. Select the process with the **lowest pass value**.
2. Run it for one quantum.
3. Update its pass value:

$$pass_i = pass_i + stride_i$$

4. Return the process to the scheduling queue.

Main Difference

Lottery scheduling achieves proportions probabilistically, whereas stride scheduling achieves them deterministically.

Stride Scheduling Example

- Tickets:
  - A: 100
  - B: 50
  - C: 250
- Using 10,000 as the constant:

$$stride_A = 100$$

$$stride_B = 200$$

$$stride_C = 40$$

- The process with the lowest pass runs next.

Pass(A) (stride=100)	Pass(B) (stride=200)	Pass(C) (stride=40)	Who Runs?
0	0	0	A
100	0	0	B
100	200	0	C
100	200	40	C
100	200	80	C
100	200	120	A
200	200	120	C
200	200	160	C
200	200	200	...

Figure 9.3: **Stride Scheduling: A Trace**

Source: chapter page 7.

Lottery vs. Stride Scheduling

- Lottery Scheduling
  - probabilistic;
  - simple;
  - little global state;
  - easy to incorporate new processes.
- Stride Scheduling
  - deterministic;
  - achieves exact proportions at the end of scheduling cycles;
  - requires maintaining pass values.
- Adding a new process is more complicated in stride scheduling because an appropriate initial pass value must be chosen.

Linux Completely Fair Scheduler

- The Completely Fair Scheduler (CFS) implements fair-share scheduling using a different approach.
- Main goals:
  - fairness;
  - efficiency;
  - scalability.
- CFS attempts to spend very little CPU time making scheduling decisions.
- Its basic mechanism is called **virtual runtime** (*vruntime*).

CFS: Virtual Runtime

- Each running process accumulates ***vruntime***.
- In the simplest case, ***vruntime*** increases with physical runtime.
- CFS selects the runnable process with the **lowest `vruntime`**.
- The objective is to keep CPU execution distributed fairly among processes.

Scheduling Decision

Run the process that has received the least virtual CPU time.

CFS: Dynamic Time Slices

- CFS uses `sched_latency` to determine a target scheduling period.
- With n runnable processes:

$$time_slice = \frac{sched_latency}{n}$$

- Example:
  - `sched_latency` = 48 ms;
  - $n = 4$ processes;
  - each process receives approximately 12 ms.

CFS: Simple Example

- Four jobs initially share the CPU.
- Each receives a 12-ms time slice.
- After C and D finish:
  - only A and B remain;
  - each receives a 24-ms time slice.
- Time slices therefore change dynamically.

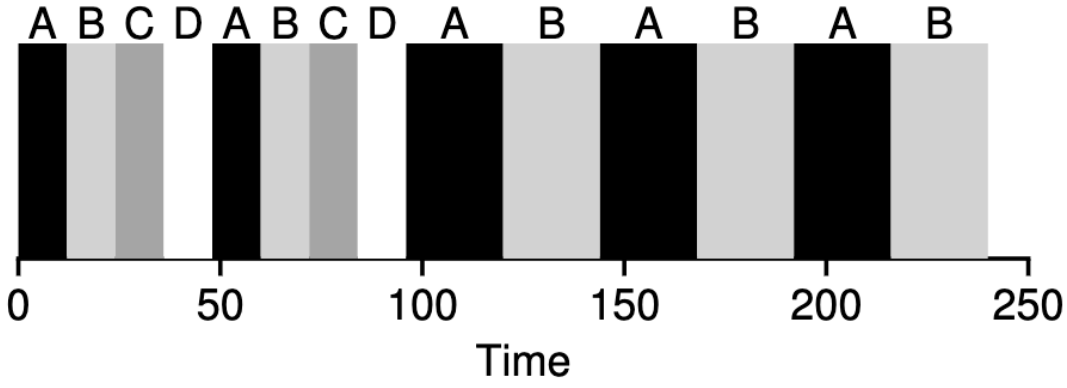


Figure 9.4: CFS Simple Example

Source: chapter page 7.

CFS: Minimum Granularity

- With many runnable processes, calculated time slices may become very small.
- Very small slices increase context-switch overhead.
- CFS therefore uses *`min_granularity`*.
- A typical value discussed in the chapter is 6 ms.
- CFS will not assign a time slice smaller than this value.

Trade-off

CFS balances short-term fairness against scheduling and context-switching overhead.

CFS: Weighting and Niceness

- CFS supports different CPU shares through UNIX **nice values**.
- Nice values range from:

$$-20 \text{ to } +19$$

- Negative nice values imply higher priority.
- Positive nice values imply lower priority.
- Nice values are mapped to numerical **weights**.

$$time_slice_k = \frac{weight_k}{\sum_{i=0}^{n-1} weight_i} \cdot sched_latency$$

Weighted Virtual Runtime

- CFS also adjusts the rate at which ***vruntime*** increases.
- Processes with larger weights accumulate virtual runtime more slowly.

$$vruntime_i = vruntime_i + \frac{weight_0}{weight_i} \cdot runtime_i$$

- Higher-priority processes therefore remain eligible to run more frequently.

CFS and Red-Black Trees

- CFS must efficiently find the process with the smallest *vruntime*.
- Runnable processes are stored in a **red-black tree**.
- Processes are ordered according to *vruntime*.
- Important operations take:

$$O(\log n)$$

- This scales better than linear lists.

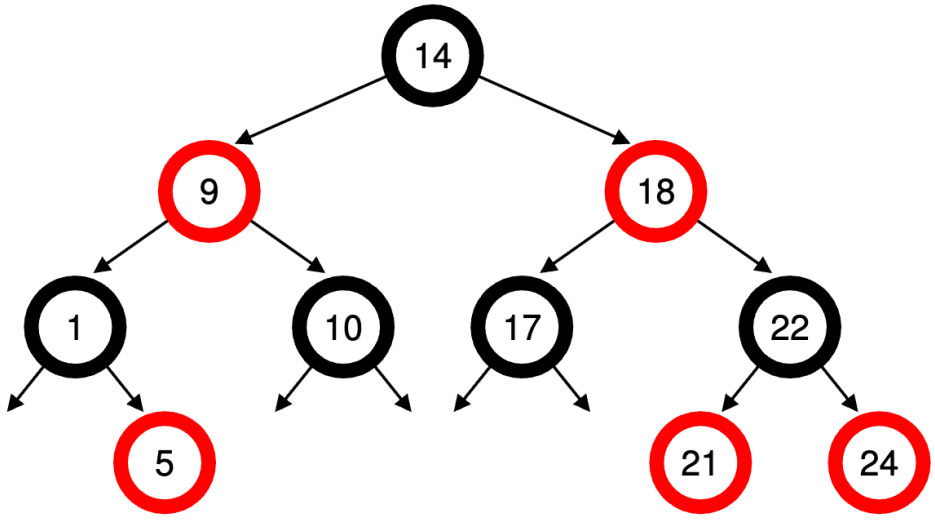


Figure 9.5: **CFS Red-Black Tree**

Source: chapter page 11.

CFS and Sleeping Processes

- Sleeping processes are removed from the red-black tree.
- A long-sleeping process may wake with a much smaller *vruntime* than active processes.
- Without correction, it could monopolize the CPU while catching up.
- CFS handles this by setting its *vruntime* to the minimum value currently found in the tree.
- This avoids starvation of other runnable processes.

Proportional-Share Scheduling: Summary

- Lottery Scheduling
  - uses tickets and randomness;
  - achieves proportional allocation probabilistically.
- Stride Scheduling
  - uses stride and pass values;
  - provides deterministic proportional allocation.
- CFS
  - uses virtual runtime and dynamic time slices;
  - uses weights to support different CPU shares;
  - uses red-black trees for efficient scheduling.

Final Remarks

- Fair-share scheduling does not solve every scheduling problem.
- Important challenges remain:
  - interaction with I/O;
  - assigning tickets or priorities appropriately.
- Proportional sharing can be effective when explicit resource allocation is desirable, such as in virtualized environments.
- As of Linux 6.6, Linux adopted the **Earliest Eligible Virtual Deadline First (EEVDF)** scheduling approach as the default.

Multiprocessor Scheduling

Multiprocessor Scheduling (Advanced)

- Multiprocessor systems have become commonplace with the rise of multicore processors.
- Multiple CPUs do not automatically make a single-threaded application faster.
- Multithreaded applications can spread work across multiple CPUs.
- The operating system must decide how to schedule jobs across multiple processors.

Key Question

How should the OS schedule jobs on multiple CPUs? What new problems arise? Do the same old techniques work, or are new ideas required?

Multiprocessor Architecture

- CPUs use small and fast hardware caches.
- Main memory is larger but slower.
- Frequently accessed data can be stored in the cache.
- Caches exploit:
  - **Temporal locality**
  - **Spatial locality**

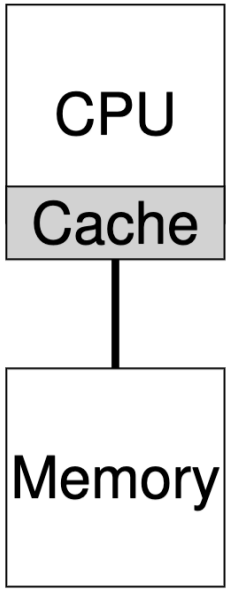


Figure 10.1: **Single CPU With Cache**

Source: chapter page 2.

Multiprocessor Caches

- Multiple CPUs may have private caches while sharing main memory.
- Different caches may contain different versions of the same memory location.
- This creates the **cache coherence** problem.
- Hardware must preserve the view of a single shared memory.

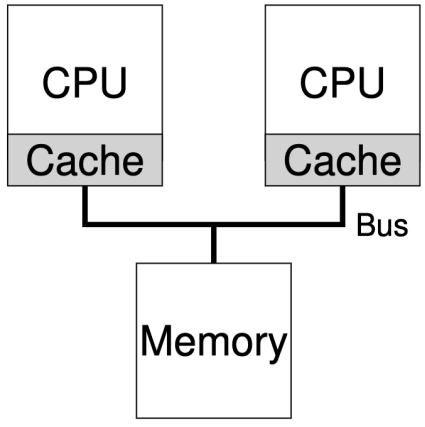


Figure 10.2: **Two CPUs With Caches Sharing Memory**

Source: chapter page 3.

Cache Coherence

- Suppose CPU 1 reads a value D from address A.
- CPU 1 modifies its cached value to D'.
- If the process later runs on CPU 2, CPU 2 may fetch the old value D from memory.
- Hardware coherence mechanisms prevent this inconsistency.

Bus Snooping

Each cache monitors memory updates by observing the shared bus. When a cache detects an update to one of its data items, it may:

- invalidate its copy; or
- update its copy.

Synchronization Is Still Necessary

- Cache coherence does **not** eliminate synchronization problems.
- Shared data structures may be accessed simultaneously by threads running on different CPUs.
- Mutual exclusion primitives, such as locks, are needed to guarantee correctness.
- Without synchronization, concurrent operations may corrupt shared data structures.

Example: Shared Linked List

```
typedef struct __Node_t {
    int value;
    struct __Node_t *next;
} Node_t;

int List_Pop() {
    Node_t *tmp = head;
    int value = head->value;
    head = head->next;
    free(tmp);
    return value;
}
```

- Two threads may read the same *head*.
- Both may try to remove the same element.
- This may cause a double *free()*.
- The same data value may also be returned twice.
- Locking can make the operation execute correctly.

Cache Affinity

- A process builds state in a CPU's caches and TLBs.
- Running the process again on the same CPU may improve performance.
- Moving the process to another CPU requires its state to be reloaded.
- A multiprocessor scheduler should therefore consider **cache affinity**.

Scheduling Goal

Prefer keeping a process on the same CPU when possible.

Single-Queue Multiprocessor Scheduling

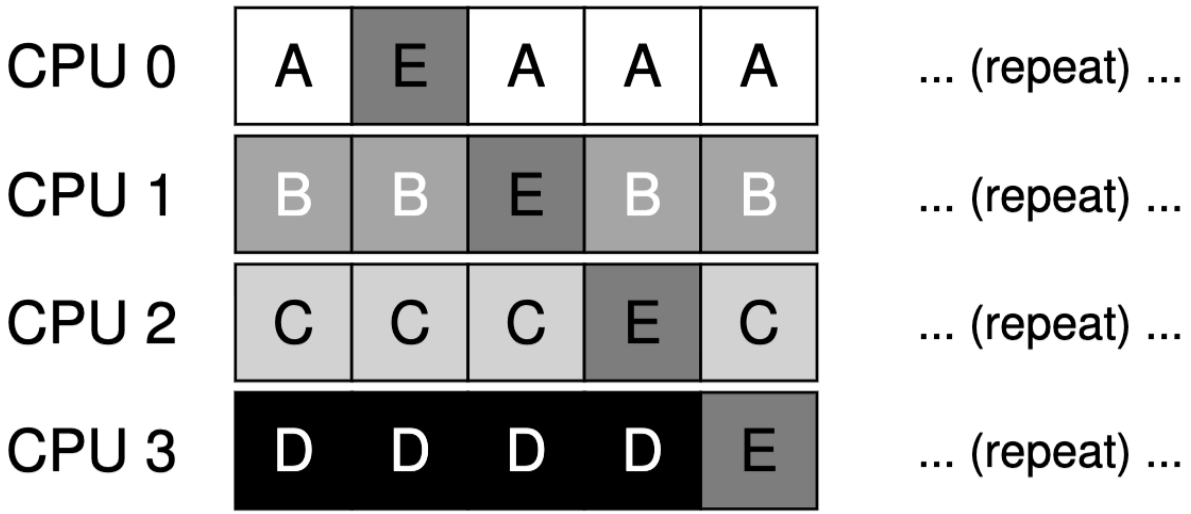
- **SQMS**: Single-Queue Multiprocessor Scheduling.
- All jobs are placed into a single global scheduling queue.
- Existing single-CPU scheduling policies can be adapted relatively easily.

Main Problems

- **Scalability**: access to the shared queue requires synchronization.
- **Cache affinity**: jobs may frequently migrate between CPUs.

SQMS and Cache Affinity

- With a global queue, jobs may move from one CPU to another.
- Frequent migration reduces the benefits of cached state.
- An affinity mechanism can keep most jobs on the same CPU.
- Some jobs may still migrate in order to balance the load.



Multi-Queue Multiprocessor Scheduling

- **MQMS**: Multi-Queue Multiprocessor Scheduling.
- Each CPU has its own scheduling queue.
- Jobs are assigned to one queue when they enter the system.
- Each queue schedules its jobs essentially independently.

Advantages

- Better scalability
- Reduced lock and cache contention
- Natural cache affinity

Load Imbalance

- Independent queues may develop different workloads.
- One CPU may have several runnable jobs while another CPU has fewer jobs.
- A CPU may even become completely idle while another CPU still has runnable jobs.

Key Question

How should a multi-queue multiprocessor scheduler handle load imbalance?

Job Migration

- Load imbalance can be addressed by moving jobs between CPUs.
- This technique is called **migration**.
- If one CPU is idle, a job can be migrated from a busy CPU.
- In more complex situations, continuous migration of jobs may be needed to maintain balance.

Work Stealing

- A queue with few jobs occasionally examines another queue.
- If the target queue is significantly more loaded, the source queue “steals” one or more jobs.
- This can improve load balance.

Trade-off

Checking other queues too frequently increases overhead and hurts scalability.
Checking too infrequently may result in severe load imbalance.

SQMS vs. MQMS

	SQMS	MQMS
Queues	Single global queue	Multiple queues
Scalability	More difficult	Better
Cache affinity	More difficult	Natural
Load balance	Easier	More difficult
Synchronization	Higher contention	Lower contention

Summary

- Multiprocessor scheduling introduces new challenges beyond single-CPU scheduling.
- Cache coherence helps maintain a consistent view of shared memory.
- Synchronization is still required when accessing shared data.
- Cache affinity favors keeping processes on the same CPU.
- SQMS is straightforward but faces scalability and affinity problems.
- MQMS scales better but introduces load imbalance.
- Migration and work stealing can help balance CPU workloads.

Summary Dialogue on CPU Virtualization

CPU Virtualization: Summary

- CPU virtualization relies on mechanisms such as **traps, timer interrupts, and context switching**.
- The OS balances **efficient execution** with maintaining **control over the machine**.
- Scheduling policies such as **SJF, RR, and MLFQ** address different performance goals.
- Scheduler design involves **trade-offs**, particularly between turnaround time and response time.
- OS design is ultimately about combining appropriate **mechanisms and policies** pragmatically.

A Dialogue on Memory Virtualization

Memory Virtualization: Summary

- Every address generated by a user program is a **virtual address**.
- The OS, with hardware support, translates virtual addresses into **physical addresses**.
- Each process sees the illusion of a **large, private, and contiguous address space**.
- Virtual memory provides **ease of use, isolation, and protection** between processes.
- Key mechanisms include **base/bounds, TLBs, and multi-level page tables**.

The Abstraction: Address Spaces

Early Systems

- Early machines provided little memory abstraction.
- The OS occupied a fixed region of physical memory.
- A single running program used the remaining memory.
- There were few “illusions” provided by the OS.

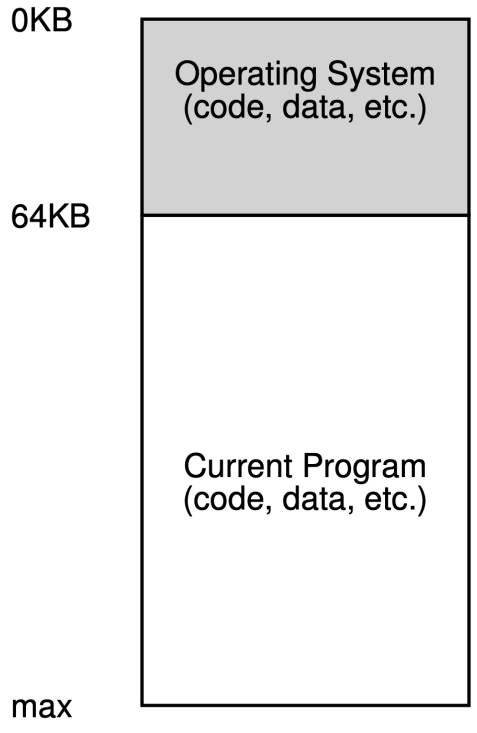


Figure 13.1: **Operating Systems: The Early Days**

Source: chapter page 2.

Multiprogramming and Time Sharing

- **Multiprogramming**: multiple processes are ready to run; the OS switches among them, e.g., during I/O.
- Main goal: improve CPU utilization.

Multiprogramming and Time Sharing

- **Multiprogramming**: multiple processes are ready to run; the OS switches among them, e.g., during I/O.
- Main goal: improve CPU utilization.
- **Time sharing**: multiple users interact with the machine and expect timely responses.
- Saving/restoring the entire memory to disk at each switch would be prohibitively slow.
- Better approach: keep multiple processes in memory and switch the CPU among them.

Sharing Physical Memory

- Multiple processes reside simultaneously in physical memory.
- With one CPU, only one process runs at a time; the others wait in the ready queue.
- This makes **protection** essential: one process must not read or modify another process's memory.

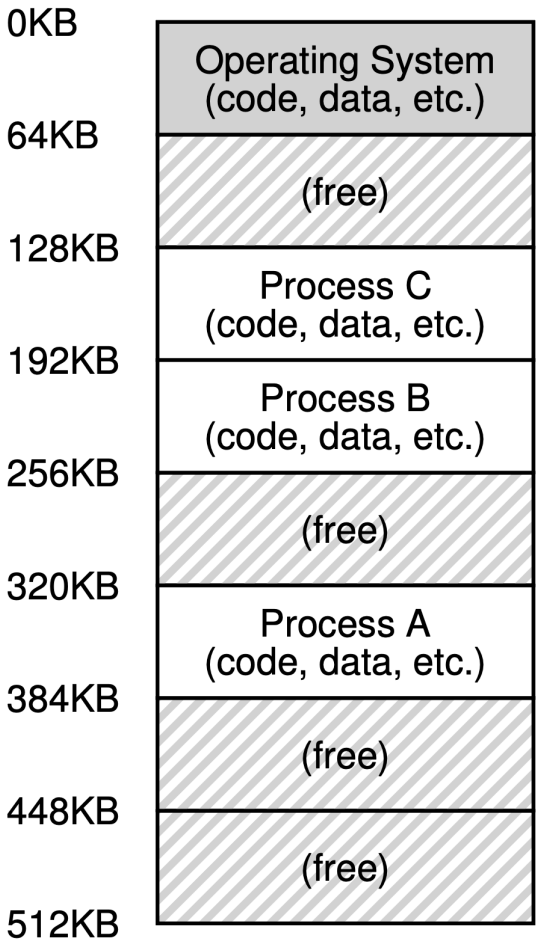


Figure 13.2: **Three Processes: Sharing Memory**

Source: chapter page 3.

The Address Space

Address Space

The running program's view of memory in the system.

- The address space contains all memory state of a program.
- Main components:
  - **Code**: program instructions;
  - **Stack**: function calls, local variables, parameters, and return values;
  - **Heap**: dynamically allocated, user-managed memory.

An Example Address Space

- Example: a small 16KB address space.
- Code is static and occupies a fixed region.
- Heap and stack can grow and shrink.
- They are placed at opposite ends to allow both regions to grow.
- This particular organization is a convention, not a requirement.

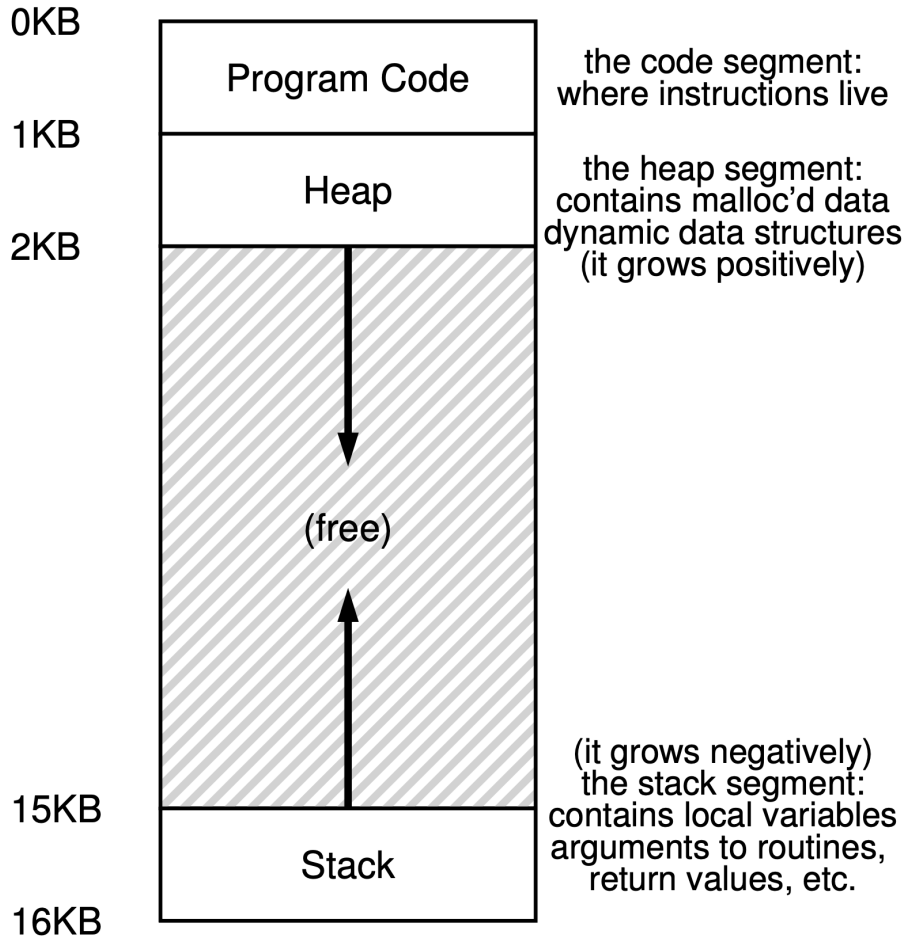


Figure 13.3: **An Example Address Space**

Source: chapter page 4.

Address Space: Abstraction vs. Reality

- The address space is an **abstraction** provided by the OS.
- A program may behave as if it occupies addresses starting at address 0.
- In reality, it can reside at arbitrary locations in physical memory.
- Thus, the address observed by a program is not necessarily the physical address used to access memory.

Compare the virtual view in Figure 13.3 with the physical placement of processes in Figure 13.2.

The Crux: How to Virtualize Memory

The Crux

How can the OS build the abstraction of a private, potentially large address space for multiple running processes, all sharing a single physical memory?

- The OS is **virtualizing memory**.
- Programs generate **virtual addresses**.
- The OS, together with hardware support, maps them to the appropriate **physical addresses**.
- Example: process A's virtual address 0 may correspond to physical address 320KB.

Goals of Virtual Memory

Transparency Virtualization should be invisible to the running program.

Efficiency Virtualization should be efficient in both time and space; hardware support such as TLBs is important.

Protection Processes must be protected from one another, and the OS must be protected from processes.

Tip: The Principle of Isolation

Isolation

If two entities are properly isolated, one can fail without affecting the other.

- Operating systems isolate processes from one another.
- Memory isolation prevents programs from affecting other processes.
- It also prevents programs from affecting the underlying OS.
- Some systems extend isolation within the OS itself, as in **microkernel** designs.

Aside: Every Address You See Is Virtual

- Any address visible to a user-level program is a **virtual address**.
- This includes addresses of code, heap objects, and stack variables.

```
printf("location of code : %p\n", main);
printf("location of heap : %p\n",
       malloc(100e6));

int x = 3;
printf("location of stack: %p\n", &x);
```

- Code appears first.
- Heap follows code.
- Stack lies near the other end of the virtual address space.

Remember

Only the OS and hardware know where these values actually reside in physical memory.

Virtual Memory: Summary

- Virtual memory provides the illusion of a **large, sparse, private address space**.
- Each address space contains the program's instructions and data.
- Programs reference memory through **virtual addresses**.
- The OS and hardware translate virtual addresses into **physical addresses**.
- The OS provides this abstraction to many processes while maintaining **protection and isolation**.
- Virtual memory requires both mechanisms and policies.

Interlude: Memory API

Memory API

The Crux: How to Allocate and Manage Memory

In UNIX/C programs, understanding how to allocate and manage memory is critical for building robust and reliable software.

- What interfaces are commonly used?
- What mistakes should be avoided?

Types of Memory

Stack Memory

- Managed implicitly by the compiler.
- Also called *automatic memory*.
- Allocated when a function is called.
- Deallocated when the function returns.

```
void func() {
    int x;
    ...
}
```

Heap Memory

- Explicitly managed by the programmer.
- Used when memory must outlive a function call.
- Allocation and deallocation require explicit calls.

```
void func() {
    int *x =
      (int *) malloc(sizeof(int));
    ...
}
```

The `malloc()` Call

Interface

```
#include <stdlib.h>
void *malloc(size_t size);
```

- Requests *size* bytes of heap memory.
- Returns a pointer to the allocated region on success.
- Returns *NULL* on failure.
- *sizeof()* is commonly used to determine the required allocation size.

```
double *d = (double *) malloc(sizeof(double));
int *x = malloc(10 * sizeof(int));
```

Using `sizeof()`

- *`sizeof()`* is an operator, generally evaluated at compile time.
- Be careful when applying it to pointers.

```
int *x = malloc(10 * sizeof(int));
printf("%d\n", sizeof(x));
```

- *`sizeof(x)`* returns the size of the *pointer*, not the dynamically allocated region.

```
int x[10];
printf("%d\n", sizeof(x));
```

- Here, the compiler has enough information to determine the size of the entire array.

Tip: When in Doubt, Try It Out

Tip

If you are unsure how a routine or operator behaves, write some code and test it.

- Manual pages and documentation are useful.
- Testing confirms how something behaves in practice.
- The chapter uses this approach to verify the behavior of `sizeof()`.

Strings and `malloc()`

- String allocation requires room for the end-of-string character.
- The suggested idiom is:

```
malloc(strlen(s) + 1)
```

- `strlen()` determines the string length.
- The additional byte stores the terminating character.
- Using `sizeof()` for this purpose may lead to errors.
- The cast of the `void *` returned by `malloc()` is not required for correctness.

The `free()` Call

- Allocating memory is relatively easy; knowing when and how to free it is harder.
- Heap memory that is no longer needed should be released with *`free()`*.

```
int *x = malloc(10 * sizeof(int));
...
free(x);
```

- *`free()`* receives a pointer previously returned by *`malloc()`*.
- The allocation size is tracked by the memory-allocation library.

Common Memory Errors

- Forgetting to allocate memory.
- Not allocating enough memory (*buffer overflow*).
- Forgetting to initialize allocated memory.
- Forgetting to free memory (*memory leak*).
- Freeing memory before use is complete (*dangling pointer*).
- Freeing memory repeatedly (*double free*).
- Calling `free()` with an invalid pointer.

Important

Compiling successfully is necessary, but far from sufficient, for a correct C program.

Forgetting to Allocate Memory

```
char *src = "hello";
char *dst;                    // unallocated
strcpy(dst, src);             // likely segfault
```

- Some routines expect the destination memory to have already been allocated.

```
char *src = "hello";
char *dst =
    (char *) malloc(strlen(src) + 1);
strcpy(dst, src);
```

- Alternatively, `strdup()` can be used.

Tip: It Compiled or Ran $\neq$ It Is Correct

Tip

A program that compiles, or even runs correctly many times, is not necessarily correct.

- Memory errors may remain unnoticed during some executions.
- Small changes can expose previously hidden bugs.
- Buffer overflows are an important example: they may appear harmless, crash the program, or create serious security problems.

Memory Leaks and Dangling Pointers

Memory Leak

- Allocated memory is never freed.
- Particularly problematic for long-running applications and systems.
- Eventually, the application may run out of memory.

Dangling Pointer

- Memory is freed before the program has finished using it.
- Subsequent use may crash the program or overwrite valid memory.

Aside: Why No Memory Is Leaked Once a Process Exits

- There are two levels of memory management:
  1. the OS manages memory assigned to processes;
  2. each process manages allocations within its own heap.
- A process may fail to call *`free()`* and thus leak memory within its heap.
- When the process exits, however, the OS reclaims all of its memory:
  - code;
  - stack;
  - heap.
- Thus, leaks are especially problematic in **long-running programs** and the **OS itself**.

More Memory Errors

Double free Calling *free()* more than once on the same memory; the result is undefined.

Invalid free Passing to *free()* a value other than an appropriate pointer previously returned by *malloc()*.

Finding Memory Problems

Tools such as *purify* and *valgrind* can help locate memory-related errors.

Underlying OS Support

- *`malloc()`* and *`free()`* are **library calls**, not system calls.
- The allocation library manages memory within the process's virtual address space.
- It relies on system calls to obtain or release memory.

`brk/sbrk` Change the end of the heap; applications should not call them directly.

`mmap()` Can create an anonymous memory region that can also be managed like heap memory.

Other Memory Allocation Calls

`calloc()` Allocates memory and initializes it to zero.

`realloc()` Creates a new, larger memory region, copies the old region into it, and returns a pointer to the new region.

Key Idea

These routines complement *`malloc()`* and *`free()`* for common dynamic-memory management needs.

Summary

- C programs use both **stack** and **heap** memory.
- Heap memory is explicitly managed by the programmer.
- *`malloc()`* allocates memory and *`free()`* releases it.
- Incorrect memory management can cause: buffer overflows, uninitialized reads, memory leaks, dangling pointers, double frees, and invalid frees.
- *`malloc()`* and *`free()`* are library calls built on top of OS memory-management mechanisms.
- Tools such as *`valgrind`* help identify memory-related problems.

Mechanism: Address Translation

Address Translation

- Memory virtualization follows the same general idea as **limited direct execution**.
- The program runs directly on the hardware whenever possible.
- Hardware support provides:
  - **efficiency** in address translation;
  - **control** over accessible memory;
  - **protection** among processes and the OS.
- Virtual memory should also provide flexibility in how applications use their address spaces.

The Crux

How can we efficiently and flexibly virtualize memory while maintaining control over application memory accesses?

Hardware-Based Address Translation

- The hardware transforms each **virtual address** generated by a process into a **physical address**.
- Translation occurs on every:
  - instruction fetch;
  - load;
  - store.
- The OS configures the hardware and manages physical memory.
- Goal: give each process the illusion of its own private memory.

Tip: Interposition Is Powerful

Hardware interposes on every memory access, transparently translating virtual addresses into physical addresses.

Initial Assumptions

To introduce address translation, assume that:

- the address space is placed **contiguously** in physical memory;
- the address space is smaller than physical memory;
- all address spaces have the same size.

These simplifying assumptions will be relaxed as more sophisticated virtualization mechanisms are introduced.

A Simple Example

Consider:

```
void func() {
    int x = 3000;
    x = x + 3;
    ...
}
```

A possible x86 translation is:

```
128: movl 0x0(%ebx), %eax
132: addl $0x03, %eax
135: movl %eax, 0x0(%ebx)
```

- Code starts around virtual address 128.
- Variable *x* is located at virtual address 15KB.
- Instruction fetches and data accesses all generate virtual addresses.

A Process and Its Address Space

- Address space size: 16KB.
- Instructions are located at addresses 128, 132, and 135.
- Variable *x* is located on the stack at address 15KB.
- From the process's perspective, all references are within its own address space.

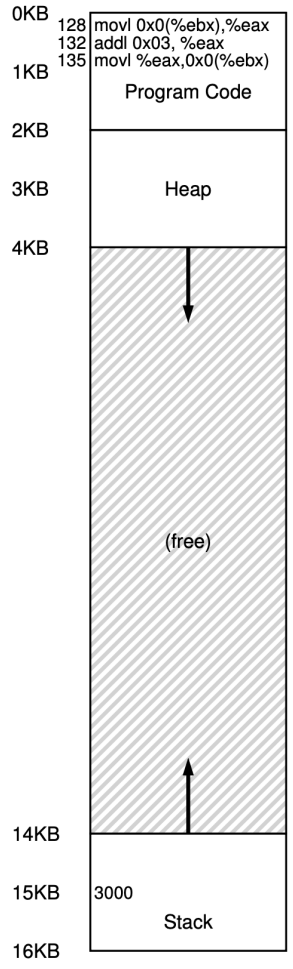


Figure 15.1: **A Process And Its Address Space**

Source: chapter page 4.

Relocating the Process

- The process believes its address space starts at 0.
- The OS may place it elsewhere in physical memory.
- In this example, the process is relocated to physical address **32KB**.
- Relocation must remain transparent to the process.

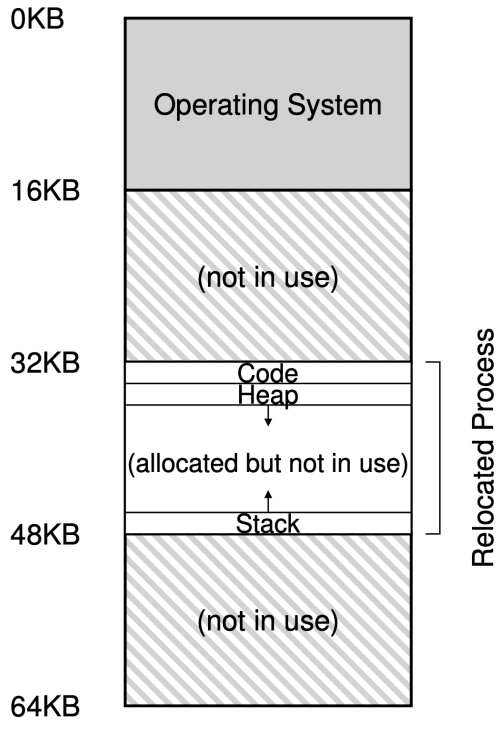


Figure 15.2: **Physical Memory with a Single Relocated Process**

Source: chapter page 5.

Question

How can a virtual address space starting at 0 be mapped to a different physical location?

Aside: Software-Based Relocation

- Early systems sometimes used **static relocation**.
- A loader rewrote addresses before execution.

$$\texttt{movl 1000, \%eax} \longrightarrow \texttt{movl 4000, \%eax}$$

- Example: the process is loaded starting at physical address 3000.
- Main limitations:
  - does not provide adequate protection;
  - makes later relocation difficult.

Dynamic Relocation: Base and Bounds

- Dynamic relocation uses two hardware registers:

  Base physical starting address of the process;

  Bounds size or limit of the address space.

- The program is compiled as if its address space starts at 0.
- The OS chooses its actual location in physical memory.

Address Translation

$$\text{physical address} = \text{virtual address} + \text{base}$$

Example: Dynamic Relocation

Assume:

$$\text{base} = 32\text{KB}, \qquad \text{bounds} = 16\text{KB}$$

- Instruction fetch at virtual address 128:

$$128 + 32768 = 32896$$

- Load from virtual address 15KB:

$$15\text{KB} + 32\text{KB} = 47\text{KB}$$

- Translation is performed automatically by the hardware for every memory reference.

Tip: Hardware-Based Dynamic Relocation

The base register provides relocation; the bounds register provides protection.

Bounds and Protection

- Before accessing memory, the processor verifies that the virtual address is legal.
- If the address is:
  - negative, or
  - greater than or equal to the bounds,

  the CPU raises an exception.
- The OS can then terminate the offending process.
- Base and bounds are maintained by the **Memory Management Unit (MMU)**.

Key Idea

Address translation provides both **relocation** and **protection**.

Example Translations

Assume a 4KB address space loaded at physical address 16KB.

Virtual Address	Physical Address
0	16KB
1KB	17KB
3000	19384
4400	Fault (out of bounds)

- A virtual address acts as an offset from the base.
- Out-of-bounds addresses generate an exception.

Hardware Support

Hardware must provide:

- privileged and user modes;
- base/bounds registers;
- address translation and bounds checking;
- privileged instructions to update registers;
- exception-handler registration;
- ability to raise exceptions.

Hardware Requirements	**Notes**
Privileged mode	*Needed to prevent user-mode processes from executing privileged operations*
Base/bounds registers	*Need pair of registers per CPU to support address translation and bounds checks*
Ability to translate virtual addresses and check if within bounds	*Circuitry to do translations and check limits; in this case, quite simple*
Privileged instruction(s) to update base/bounds	*OS must be able to set these values before letting a user program run*
Privileged instruction(s) to register exception handlers	*OS must be able to tell hardware what code to run if exception occurs*
Ability to raise exceptions	*When processes try to access privileged instructions or out-of-bounds memory*

Figure 15.3: **Dynamic Relocation: Hardware Requirements**

Source: chapter page 9.

Aside: The Free List

Data Structure: The Free List

The OS must keep track of physical-memory regions that are currently unused.

- A simple solution is a **free list**.
- It contains the ranges of physical memory that are currently available.
- When a process is created, the OS searches the free list for space.
- When a process terminates, its memory is returned to the free list.

Operating System Responsibilities

The OS must:

- allocate memory when a process is created;
- reclaim memory when a process terminates;
- manage the free list;
- save and restore base/bounds during context switches;
- install and handle exceptions.

OS Requirements	Notes
Memory management	*Need to allocate memory for new processes;* *Reclaim memory from terminated processes;* *Generally manage memory via **free list***
Base/bounds management	*Must set base/bounds properly upon context switch*
Exception handling	*Code to run when exceptions arise;* *likely action is to terminate offending process*

Figure 15.4: **Dynamic Relocation: Operating System Responsibilities**

Source: chapter page 10.

Dynamic Relocation and Context Switching

- Each CPU has one base/bounds pair.
- Different processes require different values.
- During a context switch, the OS:
  1. saves the current base/bounds values in the process structure or PCB;
  2. restores the values of the next process.
- A stopped process can even be moved to another physical location.
- The OS only needs to update its saved base value.
- The process remains unaware of the relocation.

Limited Direct Execution: Boot

At boot time, the OS:

- initializes the trap table;
- registers exception handlers;
- starts the interrupt timer;
- initializes the process table;
- initializes the free list.

OS @ boot (kernel mode)	Hardware	(No Program Yet)
initialize trap table		
	remember addresses of...	
	system call handler	
	timer handler	
	illegal mem-access handler	
	illegal instruction handler	
start interrupt timer		
	start timer; interrupt after X ms	
initialize process table		
initialize free list		

Figure 15.5: **Limited Direct Execution (Dynamic Relocation) @ Boot**

Source: chapter page 11.

Limited Direct Execution: Runtime

- OS configures base/bounds.
- Process A runs directly.
- Hardware translates its memory accesses.
- A timer interrupt returns control to the OS.
- OS switches to Process B.
- An illegal access by B causes an exception.
- OS terminates B and reclaims its memory.

OS @ run (kernel mode)	Hardware	Program (user mode)
To start process A:		
allocate entry in process table		
alloc memory for process		
set base/bound registers		
return-from-trap (into A)		
	restore registers of A	
	move to **user mode**	
	jump to A's (initial) PC	
		Process A runs
		Fetch instruction
	translate virtual address	
	perform fetch	
		Execute instruction
	if explicit load/store:	
	ensure address is legal	
	translate virtual address	
	perform load/store	
		(A runs...)
	Timer interrupt	
	move to **kernel mode**	
	jump to handler	
Handle timer		
decide: stop A, run B		
call `switch()` routine		
save regs(A) to proc-struct(A) (including base/bounds)		
restore regs(B) from proc-struct(B) (including base/bounds)		
return-from-trap (into B)		
	restore registers of B	
	move to **user mode**	
	jump to B's PC	
		Process B runs
		Execute bad load
	Load is out-of-bounds;	
	move to **kernel mode**	
	jump to trap handler	
Handle the trap		
decide to kill process B		
deallocate B's memory		
free B's entry in process table		

Figure 15.6: **Limited Direct Execution (Dynamic Relocation) @ Runtime**

Source: chapter page 12.

Base and Bounds: Benefits and Limitations

Benefits

- Simple and efficient address translation.
- Transparent relocation.
- Protection through bounds checking.

Limitation: Internal Fragmentation

A fixed-size address-space slot may contain large unused regions between the heap and stack.

Summary

- **Address translation** extends limited direct execution to virtual memory.
- Hardware translates virtual addresses into physical addresses on every memory reference.
- **Base and bounds** provide:
  - efficient dynamic relocation;
  - transparent virtualization;
  - memory protection.
- The OS manages physical memory, context switches, and exceptions.
- Fixed-size contiguous allocation can cause **internal fragmentation**.
- A more sophisticated mechanism is therefore needed: **segmentation**.

Segmentation

Why Segmentation?

- A single base-and-bounds pair requires the entire address space to reside in physical memory.
- Large unused regions may exist between the heap and the stack.
- **Segmentation** divides the address space into logical, variable-sized segments.
- Typical segments: **code, heap, and stack**.

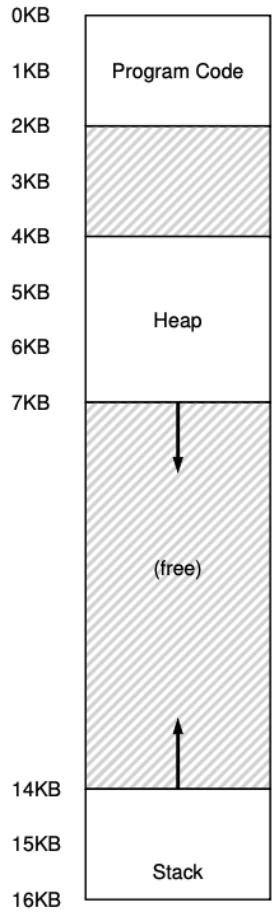


Figure 16.1: **An Address Space (Again)**

Source: chapter page 2.

Segmentation: Base and Bounds

- Each segment has its own **base** and **bounds** registers.
- Each segment can be independently placed in physical memory.
- Only used portions of the address space consume physical memory.

Segment	Base	Size
Code	32K	2K
Heap	34K	3K
Stack	28K	2K

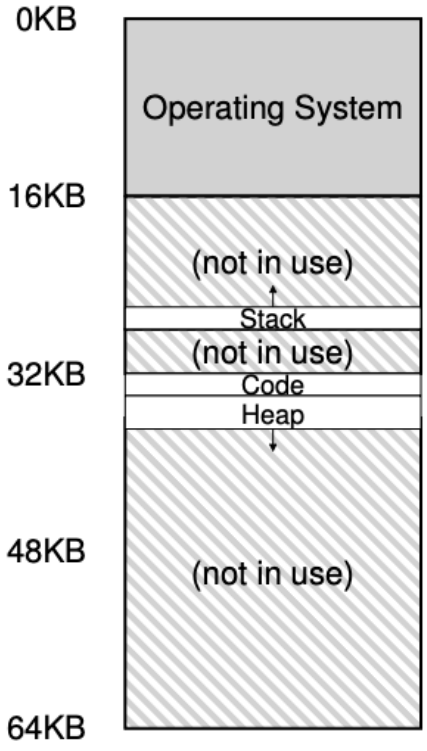


Figure 16.2: **Placing Segments In Physical Memory**

Source: chapter page 3.

Address Translation

- A virtual address identifies the **segment** and the **offset** within it.

$$\underbrace{b_{13}b_{12}}_{\text{Segment}}\ \underbrace{b_{11}\ldots b_0}_{\text{Offset}}$$

Example: virtual address 4200:

$$\underbrace{01}_{\textit{Heap}}\ \underbrace{0000\ 0110\ 1000}_{104}$$

$$PA = Base[Heap] + 104 = 34KB + 104 = 34920$$

- An out-of-bounds offset causes a **segmentation fault**.

What About the Stack?

- The stack grows toward **lower addresses**.
- Hardware records the growth direction of each segment.
- For virtual address 15KB:

$$Offset = 3KB - 4KB = -1KB$$

$$PA = 28KB - 1KB = 27KB$$

Segment	Base	Size (max 4K)	Grows Positive?
Code_{00}	32K	2K	1
Heap_{01}	34K	3K	1
Stack_{11}	28K	2K	0

Figure 16.4: **Segment Registers (With Negative-Growth Support)**

Source: chapter page 6.

Sharing and Protection

- Segments can be shared among multiple address spaces.
- Code sharing is particularly useful.
- Protection bits specify whether a segment can be:
  - read,
  - written,
  - executed.
- Read-only code can safely be shared by processes.

Segment	Base	Size (max 4K)	Grows Positive?	Protection
$Code_{00}$	32K	2K	1	Read-Execute
$Heap_{01}$	34K	3K	1	Read-Write
$Stack_{11}$	28K	2K	0	Read-Write

Figure 16.5: **Segment Register Values (with Protection)**

Source: chapter page 7.

Coarse- vs. Fine-Grained Segmentation

Coarse-Grained

- Few, relatively large segments.
- Example: code, heap, and stack.

Fine-Grained

- Large number of smaller segments.
- Requires additional hardware support.
- Segment tables can be stored in memory.

OS Support and External Fragmentation

The OS must:

- save and restore segment registers;
- handle segment growth and shrinkage;
- manage variable-sized free regions.

External Fragmentation

Free memory can become divided into small, non-contiguous regions, preventing large allocations.

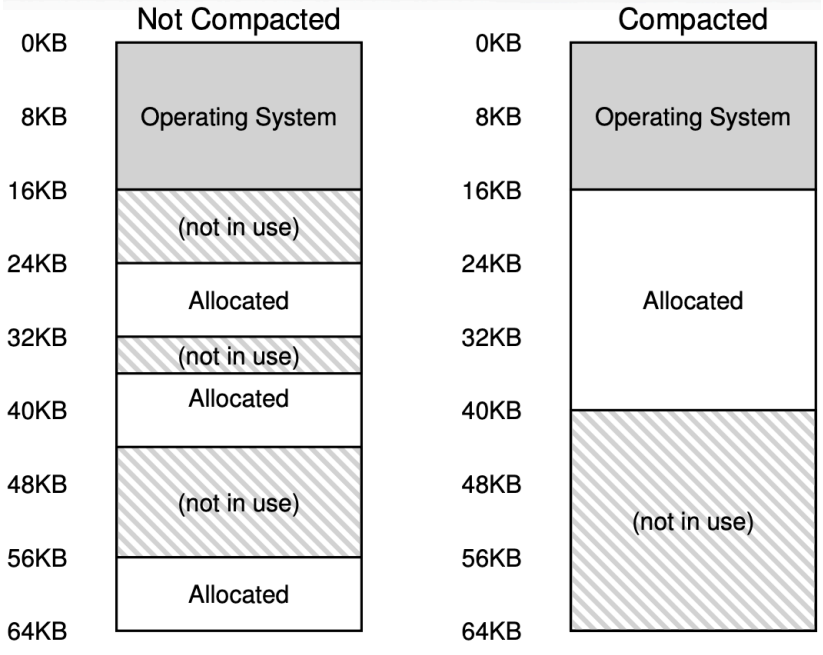


Figure 16.6: **Non-compacted and Compacted Memory**

Source: chapter page 8.

Managing Free Space

- **Compaction** rearranges segments to create larger contiguous free regions, but is expensive.
- Free-space allocation algorithms include:
  - best-fit,
  - worst-fit,
  - first-fit,
  - buddy algorithm.
- External fragmentation can be minimized, but not eliminated.

Summary

- Segmentation uses a base-and-bounds pair per logical segment.
- It efficiently supports large and sparse address spaces.
- Hardware performs relocation and bounds/protection checks.
- Segments can grow in different directions and can be shared.
- Its main limitation is **external fragmentation**.
- Segmentation is still not flexible enough for fully generalized sparse address spaces.

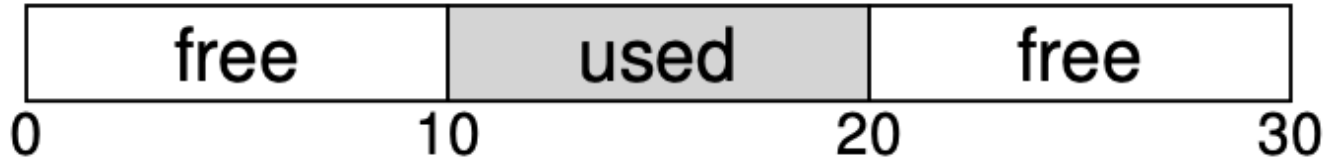
Free-Space Management

The Free-Space Management Problem

- Managing free space is simple with fixed-sized units.
- Variable-sized allocations make free-space management harder.
- The main problem is **external fragmentation**: free memory becomes divided into small, non-contiguous chunks.
- An allocation may fail even when the total free space is sufficient.

Key Question

How should free space be managed to minimize fragmentation while keeping time and space overheads low?



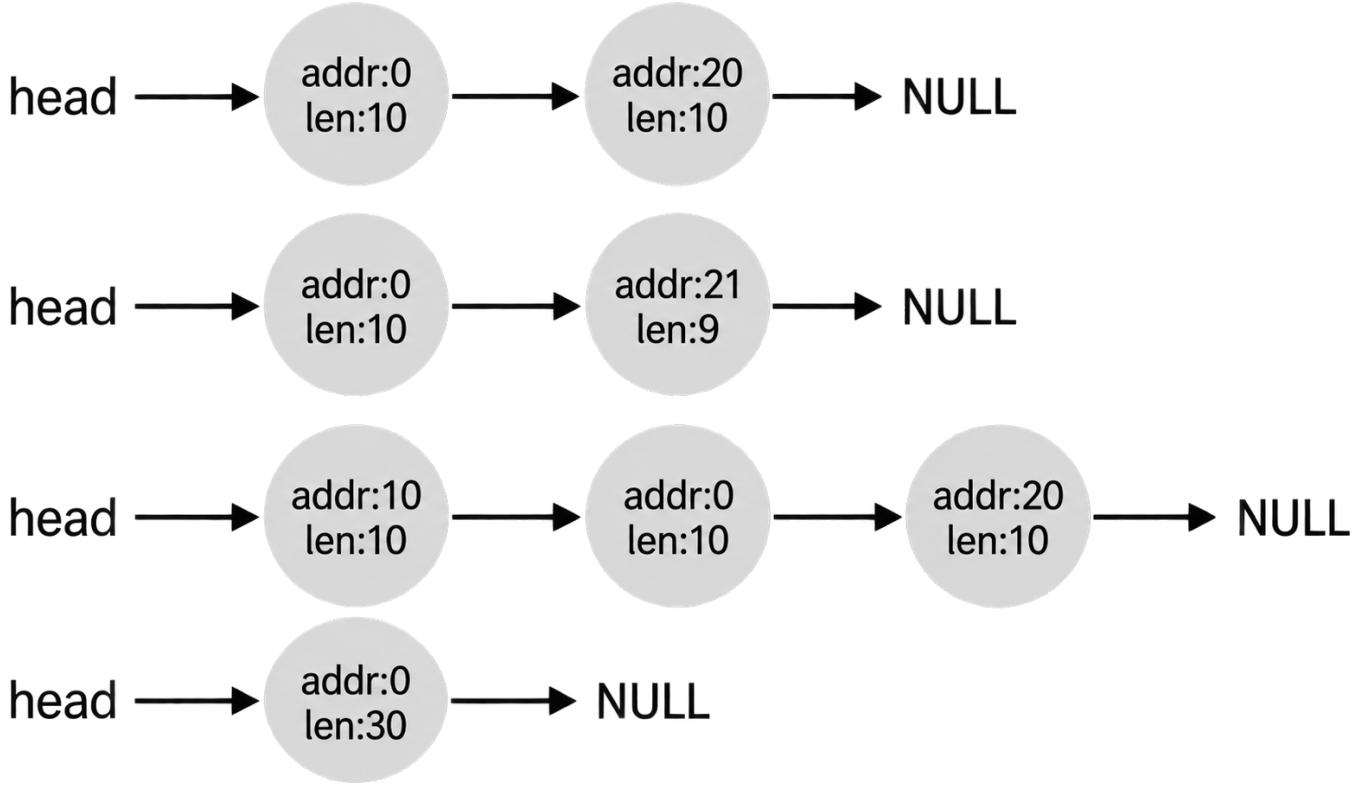
Source: chapter page 1.

Assumptions

- Focus on user-level allocators based on *malloc()* and *free()*.
- Free space in the heap is tracked through a **free list**.
- Main concern: **external fragmentation**.
- Allocated memory cannot be relocated; therefore, compaction is not available.
- Assume a single, contiguous, fixed-size heap.

Splitting

- A free list records the available chunks of memory.
- If a request is smaller than an available free chunk, the allocator can **split** that chunk.
- One part is returned to the caller.
- The remaining part stays on the free list.



Coalescing

- When memory is freed, simply inserting it into the free list can leave adjacent free chunks separated.
- **Coalescing** merges neighboring free chunks.
- This helps preserve larger contiguous regions of free memory.

Tracking Allocated Regions

- `free(ptr)` receives a pointer, but not the allocation size.
- Allocators store metadata in a **header** immediately before the allocated region.
- The header can contain:
  - allocation size;
  - a magic number;
  - additional metadata.
- Pointer arithmetic allows `free()` to locate the header.

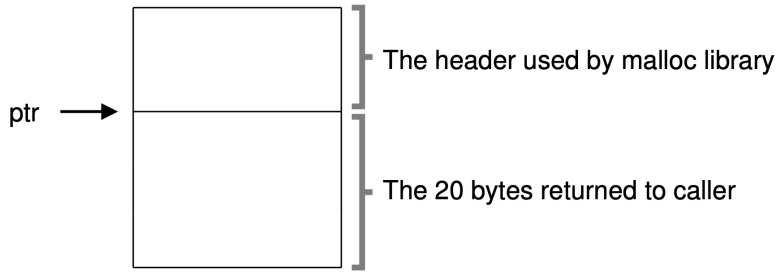


Figure 17.1: **An Allocated Region Plus Header**

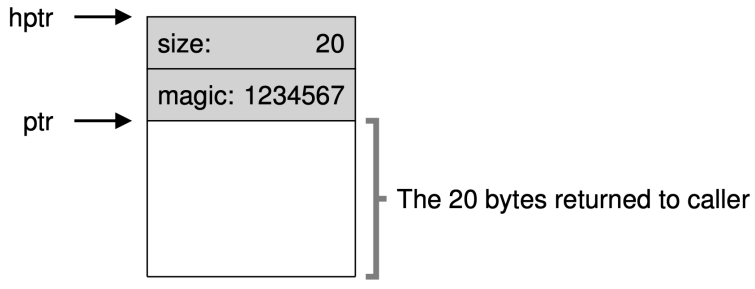


Figure 17.2: **Specific Contents Of The Header**

Source: chapter page 5.

Locating the Header

Pointer Arithmetic

```
header_t *hptr = (header_t *) ptr - 1;
```

- The header is stored immediately before the pointer returned to the application.
- The allocator can recover the allocation size from the header.
- Thus, a request for N bytes requires space for:

$$N + \text{sizeof(header)}$$

Embedding the Free List

- The allocator cannot call *`malloc()`* to create free-list nodes.
- Free-list metadata must therefore be stored **inside the free memory itself**.
- Initially, the heap can be represented by a single free node.
- The chapter illustrates this using a 4096-byte heap.

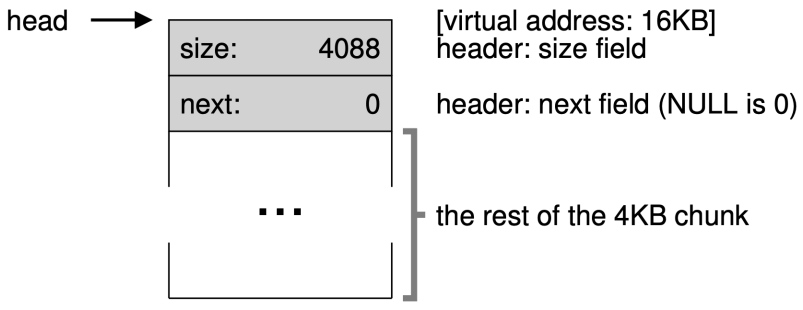


Figure 17.3: **A Heap With One Free Chunk**

Source: chapter page 7.

After One Allocation

- Suppose the application requests 100 bytes.
- The allocator selects the available free chunk.
- The chunk is split into:
  - an allocated region plus its header;
  - the remaining free region.
- With an 8-byte header, 108 bytes are consumed.

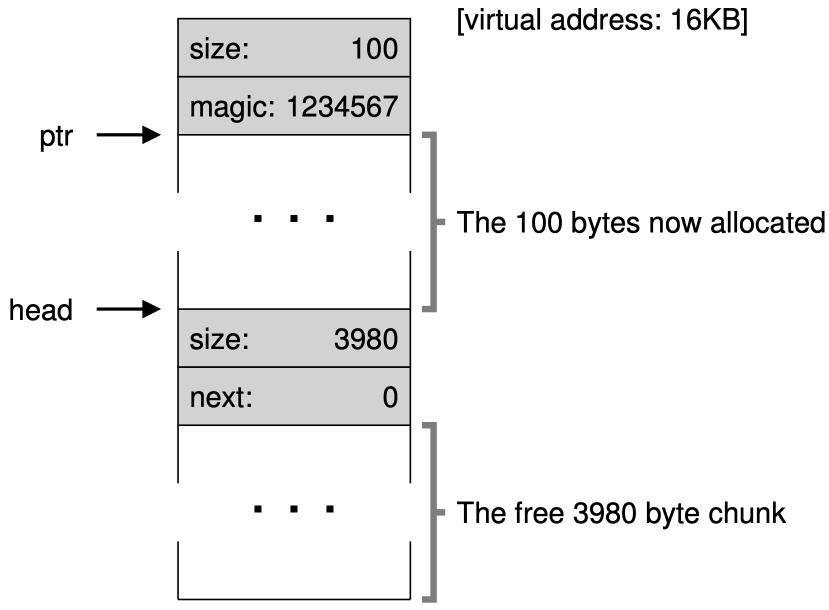


Figure 17.4: **A Heap: After One Allocation**

Source: chapter page 7.

Free-List Evolution

- Repeated allocations progressively split the free region.
- When one block is freed, it is inserted back into the free list.
- The free list may therefore contain several non-contiguous chunks.

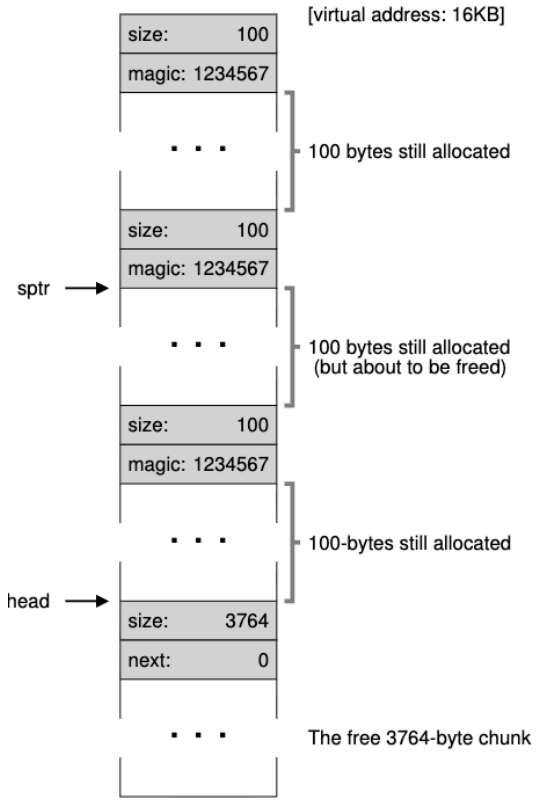


Figure 17.5: **Free Space With Three Chunks Allocated**

Source: chapter page 8.

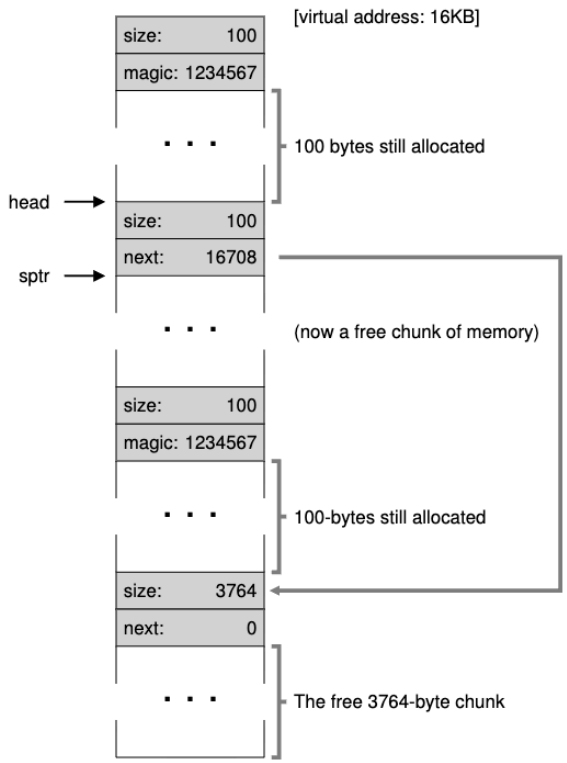


Figure 17.6: **Free Space With Two Chunks Allocated**

Source: chapter page 9.

Why Coalescing Matters

- If all allocated regions are freed without coalescing, the free list can remain fragmented.
- The heap may be entirely free but still represented as multiple separate chunks.
- Adjacent chunks must therefore be merged.

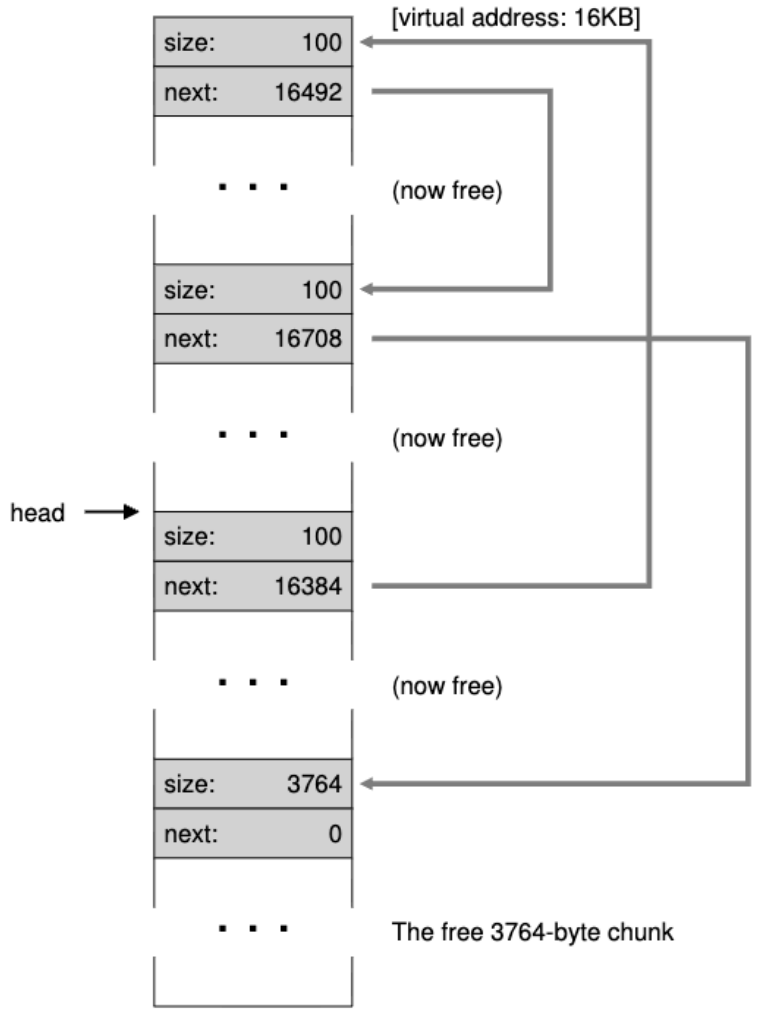


Figure 17.7: **A Non-Coalesced Free List**

Source: chapter page 10.

Growing the Heap

- If no suitable free space remains, the allocator may return *NULL*.
- Traditional allocators can instead request more memory from the operating system.
- UNIX systems historically use mechanisms such as *sbrk()*.
- The OS maps additional physical pages into the process address space, increasing the heap size.

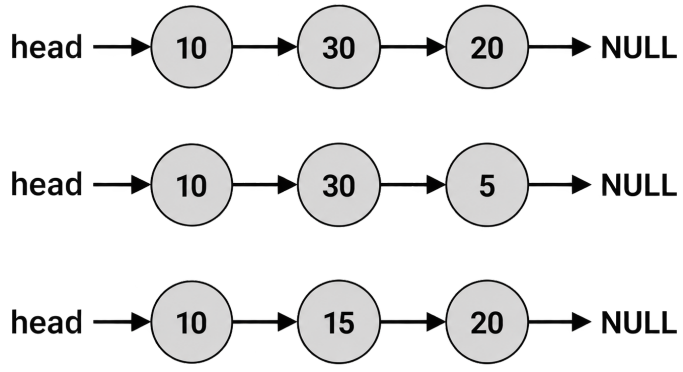
Basic Allocation Strategies

Best Fit Smallest free chunk that satisfies the request.

Worst Fit Largest available free chunk.

First Fit First chunk large enough for the request.

Next Fit Continue the first-fit search from the previous position.



- Strategies trade off search cost and fragmentation.

Segregated Lists

- Maintain separate free lists for frequently requested object sizes.
- Requests of those sizes can be served quickly.
- Fragmentation becomes less of a concern within the specialized pool.
- The **slab allocator** applies this idea to frequently used kernel objects.
- Freed objects can remain initialized, reducing initialization and destruction overhead.

Buddy Allocation

- Memory begins as one block of size 2^N.
- Blocks are recursively divided in half until a suitable block is found.
- Allocations therefore use power-of-two-sized blocks.
- When a block is freed, its **buddy** can be identified and merged efficiently.
- This simplifies coalescing, but may introduce **internal fragmentation**.

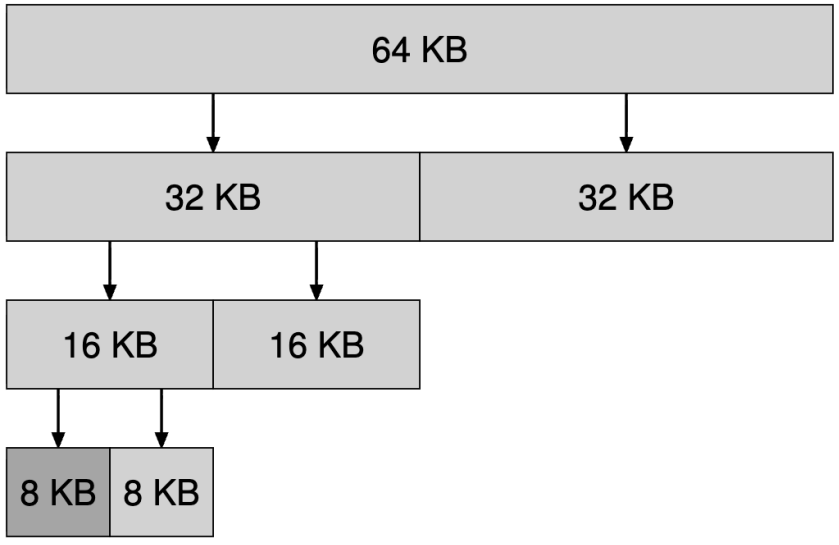


Figure 17.8: **Example Buddy-managed Heap**

Source: chapter page 15.

Other Ideas

- Simple free-list searches do not always scale well.
- More advanced allocators may use:
  - balanced binary trees;
  - splay trees;
  - partially ordered trees.
- Modern allocators must also perform well on multiprocessor and multithreaded systems.

Summary

- Variable-sized allocation introduces **external fragmentation**.
- **Splitting** and **coalescing** are fundamental mechanisms.
- Headers allow the allocator to recover allocation metadata.
- Free-list organization directly affects allocation behavior.
- Best fit, worst fit, first fit, and next fit provide different allocation policies.
- Segregated lists and buddy allocation offer alternative approaches.
- Fast, space-efficient, and scalable allocation remains a challenging systems problem.

References



Arpaci-Dusseau, Remzi H. and Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. 1.10. Arpaci-Dusseau Books, Nov. 2023. URL: https://pages.cs.wisc.edu/~remzi/OSTEP/.

? & | !